Bachelor Thesis

Zebrafish Object Tracking: A Comparative Evaluation of DeepSORT and ByteTrack

Adrian Aronsson
Computer Science

Studies from the School of Science and Technology at Örebro University
Örebro 2025

Studies from the School of Science and Technology at Örebro University

Adrian Aronsson

Zebrafish Object Tracking: A Comparative Evaluation of DeepSORT and ByteTrack

Supervisor: **Stephanie Lowry, AASS**

Examiner: **Andrey Kiselev**

Abstract

Zebrafish are widely used in laboratory research, but manual monitoring remains labor-intensive and prone to error. This thesis evaluates two different multi-object trackers, ByteTrack and DeepSORT, on their abilities to accurately track zebrafish. Integrated into a Python pipeline, the system was evaluated on a variety of aquarium video clips under different resolutions, frame rates, and lighting conditions, as well as on an annotated ground-truth sample.

Results show that motion-based tracking (ByteTrack) offers higher throughput and precision, while appearance-based association (DeepSORT) yields slightly longer continuous tracks. However, common challenges - reflections, occlusions, and erratic swimming behavior - limit both models from achieving fully reliable counting and identity maintenance in real time.

The report concludes that existing multi-object tracking methodologies are insufficient for tracking the swimming pattern of zebrafish. Enhanced detector training, more sophisticated motion models, and multi-camera or optimized lighting setups are promising directions for developing a deployable zebrafish monitoring solution.

Keywords

Multi-object tracking, Zebrafish, ByteTrack, DeepSORT, YOLOv8, MOTA, HOTA, MOT Challenge

Availability

The implementations generated from this project, including ByteTrack, DeepSORT, and the MOTChallenge benchmarking, will be made available in the git repository linked below. The repository will also contain the various video samples and tracking results included in this thesis project.

https://git-e.oru.se/adrarv221/zebrafish-tracking-evaluation.git

Acknowledgments

I would like to extend my deepest gratitude to Stephanie Lowry for supervising this project on a weekly basis. The input you provided in regard to how to structure the project, what data to analyze, and how to present acquired knowledge has been imperative to the success of this project.

Contents

Chapter 1
Introduction

The Man-Technology-Environment research centre (MTM) at Örebro University is an institution which focuses its research efforts into understanding the effects of chemicals in the environment. More specifically, they study the impacts of persistent organic pollutants, metals, and emerging pollutants on all parts of the biosphere[1]. One such pollutant, which is being studied by MTM, are Per- and polyfluoroalkyl substances, also known as PFAS or "forever chemicals". PFAS can be found all around us in our daily lives, such as in Teflon pans[2], water-repellent clothing, and some cosmetics[3], and exposure to PFAS can, among other things, lead to increased cancer risk, increased cholesterol levels, and changes in liver enzymes[4].

In order to conduct their research on PFAS, MTM exposes zebrafish to the substances. The zebrafish (Danio rerio) is a small freshwater fish that is popular for research purposes because their embryonic development is very rapid and their embryos are relatively large, robust and transparent, and able to develop outside of their mother[5]. As such, MTM hosts several aquatic tanks that hold zebrafish for exposure to PFAS and for continuous monitoring.

The laboratory setup consists of a number of 8-liter aquatic tanks that each can hold up to 40 zebrafish. The maintenance and monitoring of the zebrafish is currently performed using manual procedures, which can include feeding, counting, and visually inspecting the zebrafish for behavioral changes and health complications. Some of these procedures are also required to be performed on weekends.

In an effort to automate some of these manual procedures, the Centre for Applied Autonomous Sensor Systems (AASS) at Örebro University has suggested the introduction of automation measures with the aim of reducing human intervention in maintaining and monitoring the zebrafish. One such automation was a proposed fish feeding system that could detect how many zebrafish were in the aquatic tank and then dispense an accurate amount of fish food based on the number of fish[6]. A further suggestion is to introduce a system that can continuously detect and track the zebrafish in real time. By implementing a system that accurately achieves these two attributes, the need for manual counting of the zebrafish would be eliminated or greatly reduced. Furthermore, if a system is developed that can accurately identify individual zebrafish and maintain their identities as they are being tracked, the system can in the future be developed further by introducing behavior analysis algorithms, which would eliminate additional human tasks.

1.1 Problem Formulation

The overall objective of this thesis is to implement an object tracking model that can identify and track zebrafish in an enclosed laboratory aquatic tank, using a video stream as the sole input. The ambition is not to immediately implement a solution that will completely alleviate the MTM staff of their manual tasks, but rather that this project can lay the groundwork for bringing such a solution into fruition in the future. In regards to this project, this involves evaluating how capable modern object trackers are at tracking small and erratic fish, and analyzing what aspects must be changed or added for a future implementation to be satisfactory.

To achieve the objective of this thesis, the following subproblems must be first achieved.

- **Object detection evaluation** - Identify which zebrafish detection model can be utilized in this project. Primarily, an evaluation of whether the YOLOv8 detection model developed by Storm (2024)[6], which has been pretrained on zebrafish, can be used.
- **Object tracker evaluation & implementation** - Decide which object tracking model(s) to implement. Important factors to take into consideration include accuracy, reidentification, and speed.
- **Performance evaluation** - Assess the performance of the object tracking models in key metrics in order to understand how well they perform in regards to the manual tasks they aim to replace. Define possible next steps for improving the model.

Regarding the final subproblem, to ensure that the solution aligns with the laboratory's practical needs, the following user requirements, as expressed by the laboratory technicians, will be specifically analyzed:

- **Counting of fish** - How capable is the system of maintaining an accurate count of the zebrafish in the tank throughout the recording?
- **Identity preservation** - How capable is the system of maintaining the identity of individual fish over time?
- **Real-time performance** - At what frame rate is the end-to-end pipeline including detector and tracker capable of running at?

1.2 Division of Work

This project and thesis has been conducted and written by the author, Adrian Aronsson, in its entirety. The software created within this project has also been written by the author. Certain parts of the software has utilized the YOLOv8-Zebrafish object detection model that was created by Elliot Storm in his previous bachelor's thesis on detecting zebrafish using computer vision[6].

1.3 Outline

The remainder of this thesis follows the structure outlined in the following.

- **Chapter 2** gives an overview of relevant previous works in the field of multi-object tracking and object detection, while also providing background into relevant evaluation metrics for these technologies.
- **Chapter 3** details all aspects of how this thesis was carried out from a practical point of view. It covers technical implementations, video materials, evaluation methods, and more.
- **Chapter 4** comprehensively presents the results generated by running the tracker models on the gathered video samples. The chapter concludes with a discussion of the findings.
- **Chapter 5** offers some reflections on the fulfillment of the project goals, ethical and social impacts, possible future work within this field, as well as the author's own reflections on the progress of this project.
- **Chapter 6** lists all references that are cited within the thesis.

Chapter 2
Related Works

The field of computer vision, and its two subcomponents object detection and object tracking, which are the two main components in this thesis, have a rich history with extensive research having been conducted. These technologies are used today in a wide range of different applications, where fish monitoring is no exception. However, when it comes to zebrafish specifically, research is more sparse with only a handful of identified publications.

In this chapter, a brief history of multi-object tracking using computer vision will first be presented. The chapter will then present a comprehensive review of the two object tracking models selected for this study, ByteTrack and DeepSORT, and then continue with a presentation of previous research in regard to object tracking of zebrafish specifically. Lastly, the chapter will cover the relevant industry-standard object tracking metrics used in this thesis. First, however, two important algorithms used in many object trackers, and therefore also referenced frequently further in the related works chapter, are introduced in order to provide context.

2.1 Kalman Filter

Introduced in 1960 by Rudolf E. Kálmán, the Kalman filter is a real-time, recursive algorithm for estimating the hidden state of a linear dynamical system from noisy measurements, in a way that minimizes the mean-squared estimation error[7]. Today, Kalman filters are implemented in applications in numerous areas, such as orbit calculations, digital image processing, microeconomics, and target tracking[8]. When it comes to many of the object trackers discussed in this paper specifically, Kalman filters are in many cases implemented to provide predictions of where tracks from the previous frame will be in next frame.

2.2 Hungarian Algorithm

Published by Harold Kuhn in 1955, the Hungarian method introduces the first polynomial-time algorithm that can optimally assign n agents to n tasks in order to minimize the total

cost. The algorithm achieves this by arranging a cost table with rows representing agents and columns representing tasks. Then, for each row, the smallest value is deducted from each entry. The same is done for each column. The various combinations of n cells that are zero and do not share the same row or column are optimal solutions[9]. When it comes to many of the object trackers discussed in this paper, the Hungarian algorithm is utilized to find minimum-cost assignments between predicted tracks and detections.

2.3 Historical Background of Object Tracking

The origins of computer-vision-based object tracking date back to the introduction of the Kalman filter in the 1960's. The Kalman filter (KF) is a linear, minimum-variance Bayesian estimator using a two-step recursion, which enables computationally efficient and real-time object tracking[8]. The early success of KF in radar and video surveillance laid the foundation for application of KF in modern trackers. However, its reliance on Gaussian noise and linearity quickly exposed its limitations when dealing with non-linear appearance changes, which prompted the development of improved versions such as the Extended and Unscented Kalman filters[8].

To address the shortcomings of Kalman filters, the CONDENSATION (Conditional Density Propagation) algorithm was introduced by Isard and Blake (1998)[10], which was based on particle filtering. Instead of relying on closed-form state propagation, CONDENSATION represents the state distribution using a set of weighted samples - multiple guesses about where the object might be, each updated over time. This method allows for better results in cluttered environments where multiple competing hypotheses have to be handled. Most importantly, the authors showed that CONDENSATION was effective in tracking flexible shapes, such as moving people, and that the algorithm could recover from temporary occlusions.

During the same period as particle filter methods were introduced, appearance-based tracking methods also emerged during this time. One of the more prominent contributions in this category was the mean-shift tracking algorithm which was introduced by Comaniciu et al. (2000)[11]. Their algorithm utilizes a color histogram to model the tracked object and to find it in the next frame by maximizing the similarity between the original model and candidate regions. It does so by using a non-parametric density estimation technique to measure histogram similarities, and applies mean-shift iterations to efficiently find the best match. Unlike previously covered tracking models, which rely on motion-related cues, mean-shift tracking instead relies on appearance-based attributes for its tracking. This makes it favorable for tracking objects that are non-rigid, partially occluded, or otherwise distorted.

The next notable breakthrough in object tracking emerged in the early 2010's - namely discriminative correlation filters (DCFs). The main difference that DCFs introduced was that they framed the tracking as a classification problem, essentially learning a filter that distinguishes the object from its background. The Minimum Output Sum of Squared Error (MOSSE) tracker was the first to show that this method could achieve impressive speeds of up to 669 frames per second using simple grayscale templates[12]. The Kernelized Correlation Filter (KCF) (2014) was later introduced and extended the MOSSE algorithm with multi-channel features and kernel tricks in order to improve accuracy while maintaining real-time performance[13]. Both of these trackers were lightweight, accurate, and very efficient, making them attractive for lightweight systems such as mobile and embedded systems.

With the rise of deep learning in the mid-2010's, new breakthroughs also followed in the field of object tracking. A notable development was the introduction of Siamese network-based tracking, with SiamFC (Siamese Fully-Convolutional Network) (2016)[14] being a prominent model. This novel method reframed tracking as a template tracking problem where the object from the first frame is embedded as a reference, and then compared and matched against regions in later frames. This matching is performed using a shared convolutional neural network. Instead of using hand-crafted features, SiamFC used deep features, which helped the model achieve robustness in tracking despite appearance changes, occlusion, and deformation in the objects. It is also efficient and is capable of running at up to 86 frames per second[14].

The most recent era of object tracking models can be said to have started around 2020 with the introduction of transformer-based trackers. Notable contributions using this architecture include TransT (2021)[15] and TransTrack (2020)[16], which introduced the use of attention mechanisms, allowing them to more effectively understand how objects move and change over time. Another impactful tracker from this era is OSTrack (2022)[17], which simplified tracking by using a single vision transformer instead of the older two-branch Siamese design. This design change greatly improved both the accuracy and simplicity of the model. Not only did these transformer-based models achieve state-of-the-art performance on many object tracking benchmark datasets, but they also consolidated the previously separate tasks of detecting and tracking.

Currently, transformer-based trackers, such as those mentioned above, set new state-of-the-art performance by integrating detection and association inside a single attention-based network. However, in many laboratory or resource-constrained environments, such as the zebrafish tracking problem in this context, simpler "detector + Kalman filter + Hungarian algorithm"-pipelines remain popular because they require less specialized hardware and smaller training sets. In particular, recent works such as ByteTrack (Zhang et al., 2022)[18] and DeepSORT (Wojke et al., 2017)[19] offer a balance between accuracy, speed, and adaptability. The next sections will focus on these two frameworks in depth.

2.4 ByteTrack

ByteTrack, which was originally proposed in 2022 by Zhang et al.[18], introduces a new two-stage data-association framework for object tracking. It does this by valuing every detection box instead of discarding low-confidence detection boxes. Previously, tracking-by-detection models had linked only high-confidence detection boxes across frames, which means that creating and maintaining tracks only occurred when the detector confidence exceeded a certain threshold. This could often result in missed targets or fragmented trajectories. ByteTrack's approach instead looks at all detections, regardless of confidence score, and divides these into high-score and low-score detections based on a predefined threshold.

For each frame, ByteTrack first applies a Kalman filter[7] to predict the positions of existing tracks in the new frame. A first association is then performed where the predicted tracks are matched to the high-score detections. This is done via IoU matching. Hungarian algorithm[9] is applied to finish the matching based on this similarity score. High-scoring predictions that do not result in a match are given a new track. In the second association, the remaining, unmatched tracks are matched against low-score predictions solely based on IoU overlap[18]. This helps recover objects whose detector scores dropped, while also filtering out false background boxes.

In order to maintain lost identifies over longer occlusions, ByteTrack keeps lost tracks alive for up to 30 frames[18], which allows for objects to be reidentified when they appear again.

The design of ByteTrack has enabled it to perform very well in benchmark multi-object tracking challenges such as MOT17 and MOT20. In these challenges, the model has produced state-of-the-art results in tracking-related metrics, as well as frame rate performance[18].

2.5 DeepSORT

DeepSORT, introduced by Wojke et al. in 2017[19] is an object tracking model that builds on the existing Simple Online and Realtime Tracking (SORT) framework, which was introduced by Bewley et al. in 2016[20]. SORT is a lightweight tracking-by-detection framework that utilizes classical motion and association techniques to achieve its tracking results. At each frame, SORT takes bounding box detections from a detector as input, and will then represent each tracked object's state using a vector:

$$\mathbf{x} = \left[u,\ v,\ s,\ r,\ \dot{u},\ \dot{v},\ \dot{s}\right]^{T}$$

In this state vector, (u, v) represents the box' center, s the scale, r the aspect ratio, and the dots represent their respective velocities. A constant-velocity Kalman filter[7] is used to model motion, which helps the tracker predict the new state of each track. In order to associate data, SORT calculates the IoU between each predicted track box and the detection boxes, and then solves the assignment using the Hungarian algorithm[9]. Any IoU matches under a certain threshold are discarded[20].

DeepSORT builds on top of this by integrating a learned appearance model in order to handle occlusions and thereby reduce identify switches. In the tracking process, each detection is represented both by its bounding box Mahalanobis distance and by the cosine distance between its current feature vector and the track's gallery of recent feature vectors[19]. The Mahalanobis distance is a measure of how many standard deviations a point is from the mean of a distribution, accounting for correlations between variables[21].

Associations are handled using a matching cascade, in which young tracks are prioritized for matching first. The age of the track is simply for how long it has been unmatched. This prioritization is done to give less weight to old and uncertain tracks. Once all age groups up to the maximum age have had their chance, a final pass using regular IoU matching is done on all unmatched tracks and detections[19]. The maximum age for a track is, in this project, set to 30 frames, and a new track is only created if an object is detected for 5 consecutive frames.

The improvements introduced with DeepSORT reduce identity switches by about 45% compared to SORT, while preserving the same accuracy and being able to run in real time[19].

2.6 Object Tracking of Zebrafish

Being able to detect and track zebrafish using computer vision has increasingly become more important to do accurately, in large part due to the favorable characteristics that the fish has as a research subject. The ability to count and track individual fish is a challenge that has attracted more and more research during the 2000's. In this section, some prominent contributions will be presented.

One prominent and straightforward method to track zebrafish was introduced by Xia et al. (2016) where they introduced tracking based on the posture of the fish, so both the appearance of the head and the tail. Each candidate match between a predicted track and a detection is first filtered after displacement and change in posture angle. If the match exceeds predefined thresholds for these values, they are discarded. Using their method, the authors were able to show a significant reduction in ID swaps compared to previous methods[22]. In a very similar fashion, de Oliveira Barreiros et al. (2021) utilized a convolutional network, YOLOv2, to define the heads of the fish in order to improve individual fish detection. In addition, they employed a Kalman filter in order to predict the movement of the fish' head. However, their method involved filming only from above and placing the tank above a flat LED-panel in order to improve the contrast to the fish[23]. Xu & Cheng (2017) performed a very similar project in which they also used a convolutional neural network for zebrafish identity classification. They also used a top-down mounted camera filming a shallow (5 cm) fish tank sitting on top of a LED-panel for brightness. The system first detects head regions via scale-space blob detection and extracts orientation-normalized patches, which are matched across frames to form short trajectory segments. These high-confidence segments are then used to iteratively train the neural network that classifies head patches. Once classified, segments are linked to full trajectories based on spatiotemporal proximity. Using this method, the authors were able to achieve very high precision while also keeping identity-switch rates to as few as 6-48 switches over thousands of frames[24].

In 2014 Pérez-Escudero et al. introduced idTracker, which also involved zebrafish. The study was carried out on several different species, with zebrafish being one of them. The core idea with idTrack is that it computes a characteristic fingerprint of each individual's appearance features and is then able to match detections to the stored appearance fingerprints, which helps prevent errors. idTracker was shown to be able to maintain correct identities when two nearly identical zebrafish crossed paths[25]. idTracker was later improved to idTracker.ai in 2019 by Francisco Romero-Ferrero et al. idTracker.ai employs two convolutional networks, one in the detector part and one in the identification part of the model. The system is able to train these networks adaptively until a user-defined accuracy threshold is reached. After this, the model is able to track individuals with high accuracy[26].

2.7 Object Tracking Evaluation Metrics

In order to properly know how to evaluate the accuracy and performance of an object tracker, it is important to have an understanding of what the current industry-standard evaluation metrics look like and what they reveal. When it comes to the evaluation of multiple-object trackers, it has historically encompassed not only whether the tracked objects are properly detected but also how well the object's trajectories and identities are being maintained over time.

In 2008, Bernardin and Stiefelhagen introduced what would become the first broadly adopted framework for evaluating multiple-object trackers. Their goal was to unify the various existing measures of tracking performance, false positives, false negatives, and identity errors, into two comprehensive metrics: Multiple Object Tracking Accuracy (MOTA) and Multiple Object Tracking Precision (MOTP)[27].

The next major development in regards to multiple-object tracking metrics came in 2016 when Ristani et al. introduced a set of identity-related metrics[28], with IDF1 being the most prominent one. Lastly, in 2020, Luiten et al. published a paper in which they introduced the Higher-Order Tracking Accuracy (HOTA) metric, which balances the emphasis on detection, association, and localization into one single metric[29].

All of the main metrics that are relevant to this project will be presented in more detail in the following sections. Additionally, the equations of some of the more detail-oriented metrics will be presented separately in the chapter where the results are presented.

Terminology for Tracking Outcomes

Before presenting the various multi-object tracking metrics used in this thesis, there is value in defining various concepts relating to how tracking results are defined and counted.

Whenever the tracker predicts that there is a fish in a frame location where there also actually is a fish, this is referred to as a true positive (TP). Instances where the tracker predicts that there is a fish in a frame location where there is no fish in reality are referred to as false positives (FP). If the tracker does not predict a fish in a frame location that actually holds a fish, this is called a false negative (FN). Lastly, an ID Switch (IDSW) occurs when a tracker wrongfully swaps object identities or when a track was lost and reinitialized with a different identity. [29][30]

2.7.1 MOTA

MOTA essentially looks at the number of errors made and then puts them in relation to how many ground-truth instances there were. The errors tracked are false positives, false negatives, and identity switches. Formally, the equation looks like this:

$$\text{MOTA} = 1 - \frac{\sum_{t=1}^{T} \left(\text{FN}_t + \text{FP}_t + \text{IDSW}_t\right)}{\sum_{t=1}^{T} \text{GT}_t} \times 100\% \tag{2.1}$$

The denominator, GT_t, represents the total number of ground-truth instances for frame t[**?**]. For example, for a video sample of 50 frames in which two objects are tracked for each frame, the total number of ground-truth instances would be 100.

MOTA is most commonly multiplied by 100 to obtain a percentage. When evaluating multiple-object trackers using the MOTA metric, a value as close to 100% as possible is preferable[27].

2.7.2 MOTP

While MOTA evaluates how many targets are being correctly tracked, MOTP instead reveals how accurately the targets are being tracked. It does so by computing the average overlap between its own bounding box prediction and the ground-truth bounding box for all frames and targets. The equation for MOTP looks as follows:

$$\text{MOTP} = \frac{\sum_{(t,i)} d_{t,i}}{\sum_{t} c_t} \times 100\% \tag{2.2}$$

In this equation, c_t represents the number of matches in frame t, and $d_{t,i}$ is the bounding box overlap of target i with its ground truth representation[27]. Once again, the result is typically multiplied with 100, and a MOTP value as close to 100% is preferable.

2.7.3 IDF1

ID Precision (IDP) indicates the fraction of computed detections whose identity label matches the ground-truth identity, while ID Recall (IDR) measures the fraction of ground-truth detections correctly labeled. The IDF1 metric balances the precision and recall values according to the following equation:

$$\text{IDF}_1 = \frac{2\,\text{IDTP}}{2\,\text{IDTP} + \text{IDFP} + \text{IDFN}} \times 100\% \tag{2.3}$$

Thus, IDF1 is the ratio of correctly identified detections over the average number of ground-truth and computed detections[28].

2.7.4 HOTA

HOTA is the newest metric of the set presented here, and aims to address the limitations of both MOTA and IDF1. It is designed to provide a balanced score between the detection and association performance of the tracker[29]. In order to calculate HOTA, DetA (Detection Accuracy) and AssA (Association Accuracy) must first be obtained.

$$\text{DetA}_\alpha = \frac{\text{DetRe}_\alpha \cdot \text{DetPr}_\alpha}{\text{DetRe}_\alpha + \text{DetPr}_\alpha - \text{DetRe}_\alpha \cdot \text{DetPr}_\alpha} \tag{2.4}$$

$$\text{AssA}_\alpha = \frac{\text{AssRe}_\alpha \cdot \text{AssPr}_\alpha}{\text{AssRe}_\alpha + \text{AssPr}_\alpha - \text{AssRe}_\alpha \cdot \text{AssPr}_\alpha} \tag{2.5}$$

Here DetRe is detection recall, DetPr is detection precision, AssRe is association recall, and AssPr is Association Precision. With DetA and AssA defined, HOTA is calculated using the following formula[29].

$$\text{HOTA}_\alpha = \sqrt{\text{DetA}_\alpha \cdot \text{AssA}_\alpha} \times 100\% \tag{2.6}$$

Chapter 3
Implementation

This chapter will present how the project was carried out, from both a practical, technical, and analytical point of view. As a first step, the chapter will present how the data was obtained. Following this, details of how the actual technical implementation of the models will follow. Lastly, the third section will cover how various object tracking-related metrics were obtained and how the performance of the trackers were evaluated both qualitatively and quantitatively.

3.1 Data collection

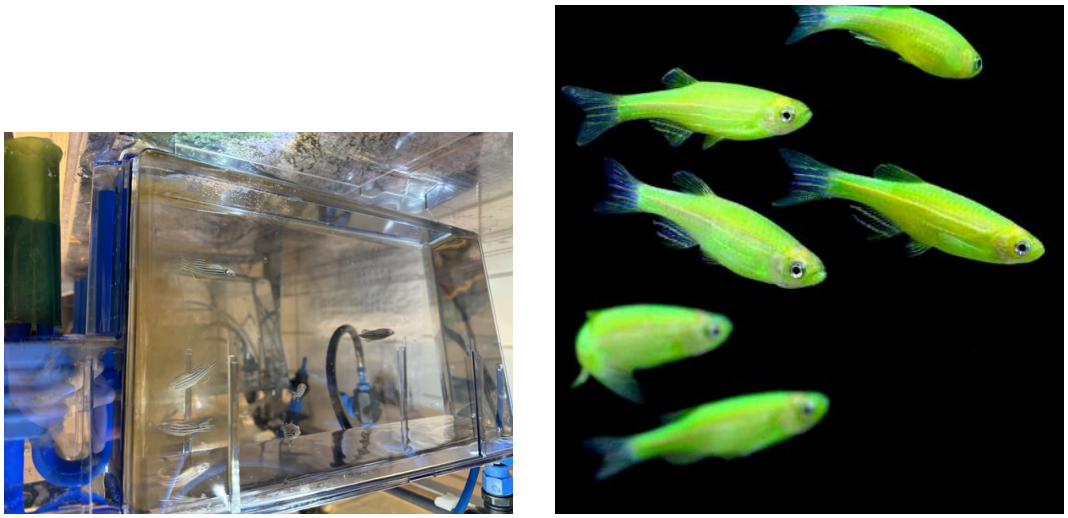
3.1.1 Data for detector training

In order for an object tracking model to function, it must rely on an object detection model that continuously feeds the tracking model detections on which it can make predictions. This means that the detection model must be trained on data that enables it to know what the target object looks like.

In the implementation for this project, the object detection model that will be applied is the "YOLO-Zebrafish2" model that was developed by Elliot Storm[6]. This model is based on the YOLOv8m architecture, which is a single-pass convolutional neural network that makes it very efficient and lightweight. The YOLO-Zebrafish2 model is already trained on 1035 images containing 2708 labels of zebrafish. Out of the 2708 total training labels, 1171 were captured with a 48MP camera in the MTM lab environment (see figure 3.1 (a)), and 946 were from a publicly available dataset of zebrafish (see figure 3.1 (b)). The YOLO-Zebrafish2 model achieved a precision of 0.92 and a recall of 0.94[6].

3.1.2 Data for tracker model evaluation

Gathering of sample videos for the evaluation of the object tracking models was carried out on two separate occasions in the MTM lab. As the object tracking models do not require any training on sample material, there is not the same need to focus on quantity when gathering data to evaluate the models, as opposed to when training a detection



(a) MTM lab sample (b) Online dataset sample

Figure 3.1: Sample images from both datasets.

Table 3.1: Summary of frame rates and resolutions for video samples.

		Resolutions
Framerate	30	1280x720 1920x1080 3840x2160 7680x4320
	60	1920x1080 3840x2160

model. Instead, the focus was on gathering different types of video samples in terms of resolution, frame rate, and lighting conditions. All video samples were recorded using a Samsung Galaxy S24+ video camera using the settings shown in table 3.1. As the laboratory only has one light setting, all video configurations were recorded both with the light on and off. With the light off, the only remaining light source was the aquarium-specific lighting. This resulted in 12 video samples. Each video sample was 30 seconds long.

The aquatic tank used for the video samples was an 8-liter tank that held 11 zebrafish. The smartphone was placed on its side, perpendicular to the long side of the fish tank, without any particular mounting.

3.1.3 Ground truth video sample

In addition to the evaluations that can be performed on the video samples covered in the previous section, there are also additional object tracking metrics that can be obtained and evaluated if the input video sample contains ground-truth annotations. A video that has

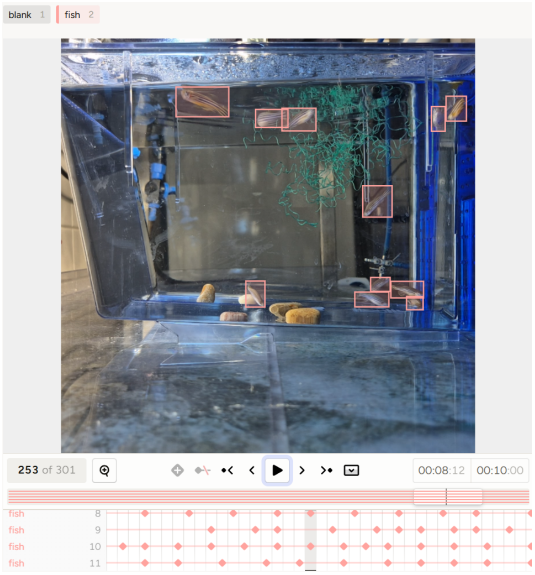


Figure 3.2: Example of a fully-annotated frame in Label Studio

ground-truth annotations has, for each video frame, attached data that clarifies where the bounding boxes for each target object is.

The ground truth labeling for this project is made using Label Studio. Label Studio is a platform on which it is possible to label and evaluate multi-modal data[31], which in this case means video. Once a video file has been uploaded to the platform, and possible labels have been defined, each target object can be labeled using a bounding box in the first frame that the target object is visible. The user is then able to move forward, frame-by-frame, until the last frame before the target object changes direction or speed, and in that frame set a new bounding box around the target subject. The Label Studio software will then make the bounding box follow the target object for the duration of the frames that the target object maintains the same speed and direction. See figure 3.2 for a depiction of what part of the platform looks like.

Because the zebrafish swim very quickly and erratically, this means that the adjustments required on the ground-truth bounding boxes come very frequently, sometimes just one or two frames after each other. Hence, the ground-truth labeling is very time consuming for video samples containing zebrafish. Due to time constraints within this project, only one 10-second video sample will be annotated with ground-truth labeling. The sample video is of resolution 1440x1440 with 30 frames per second. The video sample is of an 8-liter tank holding 11 zebrafish.

3.2 Model implementations

Two different object tracking models were implemented within this project. Details on their implementations will be covered in this section.

3.2.1 ByteTrack tracker in Python

The ByteTrack multi-object tracking algorithm was implemented in the same Python Conda environment that the object detection pipeline was developed with, which ensured that the integration of the two models could be done effortlessly. In order to install ByteTrack, its repository was cloned from GitHub[32] and installed together with various required dependencies.

Each detection frame from the YOLO-Zebrafish detector is converted to ByteTrack's "x1, y1, x2, y2, score, class" tensor and passed to the model's update method, after which the tracker's "top-left x, top-left y, width, height" (tlwh) output is rendered with its track IDs.

The Python script will write an annotated video, and logs FPS and ID statistics, enabling both qualitative and quantitative analysis from the output.

A flowchart depicting how the ByteTrack implementation works, together with the YOLO-Zebrafish2 detector, is shown in figure 3.3. The flowchart clarifies where the detector is located, how the Kalman filter provides predictions, and how the two-pass matchings are performed.

3.2.2 DeepSORT tracker in Python

The Python implementation of DeepSORT follows much of the same format as the implementation for ByteTrack. The main difference is that DeepSORT can easily be installed via a simple "pip install deep-sort-realtime" shell command and by ensuring that some dependencies are installed.

For each frame, the YOLO-Zebrafish detections are converted to the format expected by DeepSORT, which is tuples of "tlwh, confidence, class", where tlwh stands for the coordinates of the top-left corner, together with the width and height of the bounding boxes. These are passed to DeepSORT's method for updating tracks to assign consistent track IDs.

Once again, the script's output is an annotated video file, together with logs of FPS and ID statistics.

In figure 3.4, the DeepSORT implementation is shown with the YOLO-Zebrafish2 detector included. The flowchart clarifies how the appearance modeling is included in the matching logic.

3.2.3 Hardware used for tracking

Both trackers were run on the same machine with the following notable specifications.

- Windows 11 64-bit operating system
- Intel Core i7-14700KF CPU
- MSI GeForce RTX 4080 SUPER 16BG GPU
- 64GB 6000MHz DDR5 RAM Memory

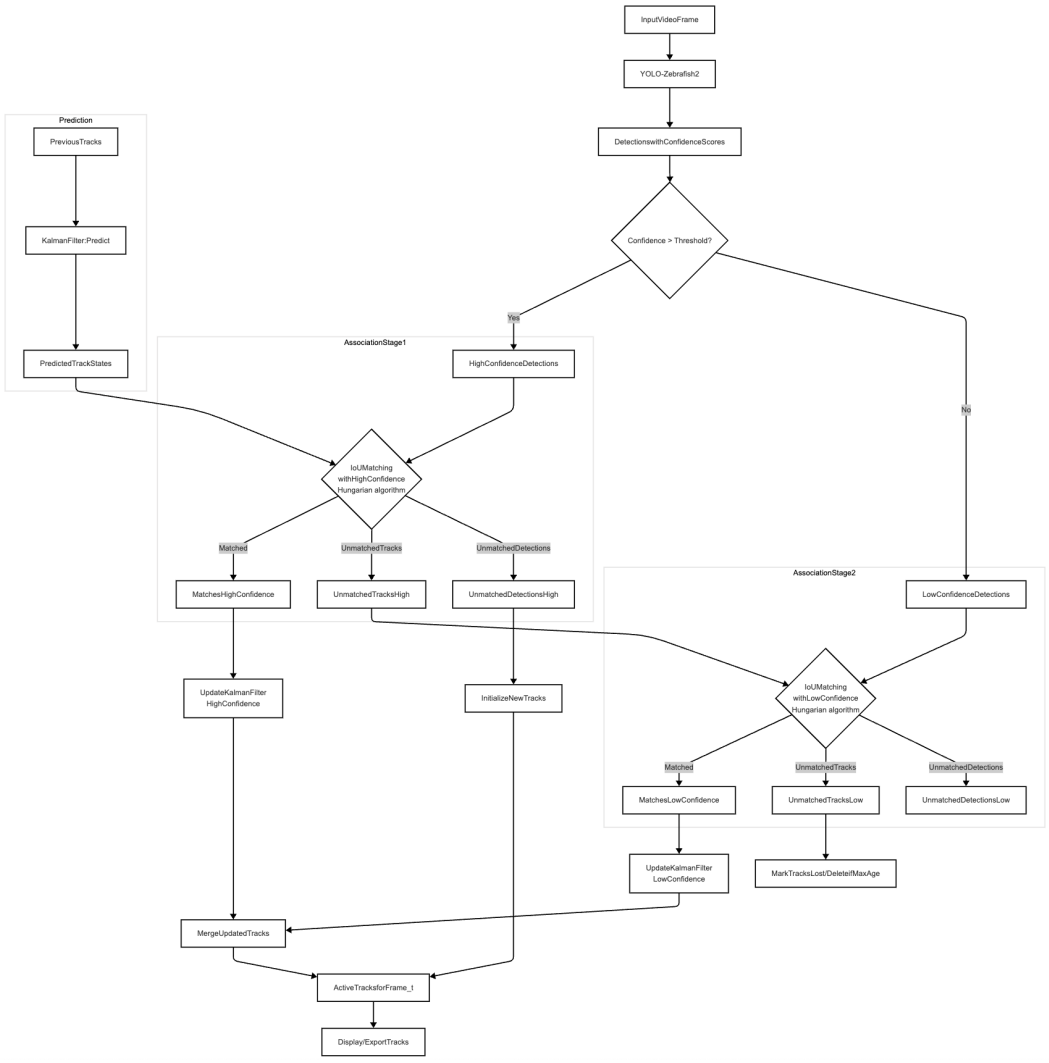


Figure 3.3: The ByteTrack flowchart with YOLO-Zebrafish2 detector included

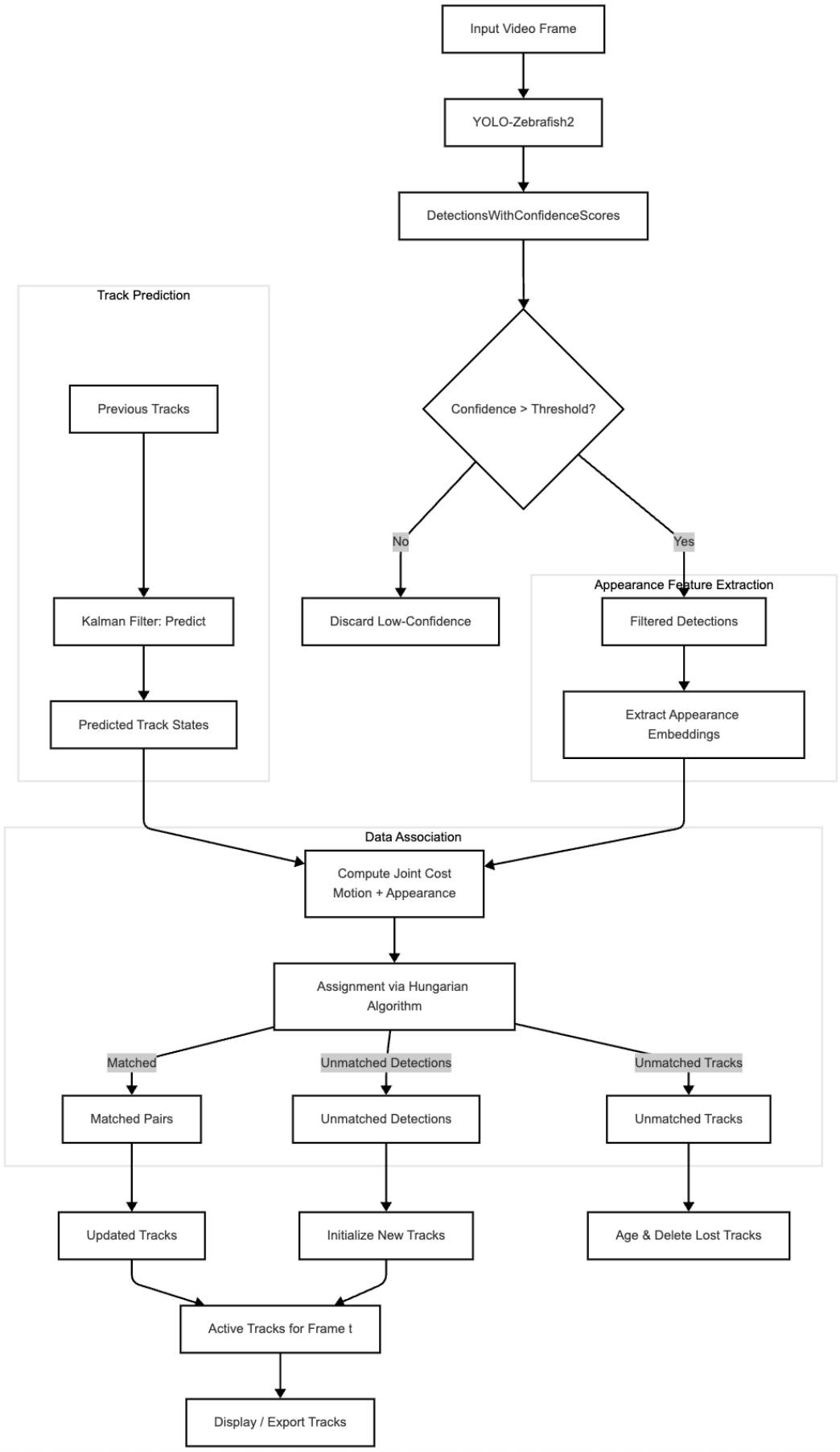


Figure 3.4: The DeepSORT flowchart with YOLO-Zebrafish2 detector included

3.3 Object tracker evaluation

3.3.1 Quantitative evaluation

To quantitatively evaluate the tracking performance of zebrafish by the trackers, two categories of metrics were obtained. The first one being metrics that can be obtained by running the tracker without any comparison to ground truth data. These metrics will from now on be referred to as the Online Tracking Metrics and the metrics included in this category are included in table 3.2 together with a brief explanation of what they show. What these metrics all have in common is that they can be easily produced and tracked directly in the regular object tracking Python scripts. After each run of the script, a printout of the mentioned metrics is made in the terminal window.

Table 3.2: Tracker Performance Metrics

Metric	Explanation
FPS	Number of frames per second that the tracker runs at.
Unique fish	Number of unique fish identified by the tracker.
Unique tracks	Number of unique tracks identified by the tracker.
Frames/Fish	Average number of frames each fish is tracked for over all associated tracks.
Frames/Track	Average number of frames each track is active for.
Tracks/Fish	Average number of tracks associated with each unique fish.
Lost fish	Number of times a fish is lost.
ReID fish	Number of times a fish is reidentified after being lost.
ReID succ rate	Share of lost fish that are successfully reidentified.

The second category of quantitative data includes metrics that can only be obtained by comparing the tracker output with ground-truth data. These metrics include HOTA, MOTA, MOTP, and IDF1. In order to implement these metrics, the official TrackEval library[33], which provides implementations of all three metrics, was installed via a "pip install trackeval" command. Next, a script was created to convert the ground-truth CSV-file from Label Studio (see Section 3.1.3) into the MOT Challenge format. The object tracker pipelines were also modified to record the tracking results in the MOT Challenge format during video processing. Lastly, a final Python script was created in which TrackEval is used to produce the tracking metrics using both the ground truth and the tracker MOT Challenge format files. For all of the metrics produced, higher values are preferred.

3.3.2 Qualitative evaluation

When it comes to evaluating the object tracking models in a qualitative fashion, all of the activities involve analyzing the annotated tracker video output, sometimes frame-by-frame. The main objectives are to provide answers to the following questions:

- Under what circumstances are each tracker generating true positives (fish identified as fish),
- Under what circumstances are each tracker generating false positives (non-fish identified as fish),
- Under what circumstances are each tracker generating false negatives (fish not identified as fish),

- Under what circumstances are each tracker able to maintain the correct ID when reidentifying a fish after losing that fish, and
- Under what circumstances are each tracker assign a new ID when reidentifying a fish after losing that fish.

The final two points can be clarified further. A tracker being able to maintain the correct ID when reidentifying a fish after losing that fish implies that the tracker thinks that it is the same fish being reidentified. A tracker that assign a new ID when reidentifying a fish after losing that fish implies that the tracker believes it is a new fish. Ideally, a tracker should always be able to reassign the correct ID when losing and reidentifying a fish.

Chapter 4
Results

This chapter presents the results that were observed within all evaluations, both quantitative and qualitative. The first section will briefly recall the data that were analyzed in the results. The following section will present the qualitative results observed in the main video samples. The third section will then present quantitative results on the online metrics, meaning those metrics that are computed while the tracking script is running. The following section covers the object tracking metrics obtained from the MOT Challenge framework. Lastly, a comprehensive discussion section will conclude the chapter.

4.1 Experimental Material

The complete set of analyzed data consists of 12 30-second raw video clips (hereafter referred to as the main video samples) and one 10-second clip with ground-truth bounding boxes (hereafter referred to as the ground truth sample).

The main video samples are:

Each resolution and frame rate configuration has been captured once with the main laboratory light off and once with the light on. This results in a total of 12 video samples.

Table 4.1: Summary of frame rates and resolutions for video samples.

		Resolutions
Framerate	30	1280x720 1920x1080 3840x2160 7680x4320
	60	1920x1080 3840x2160

The ground truth sample is a single 10-second video of 1440x1440 resolution, running at 30 frames per second. It contains frame-by-frame information about the exact location of each of the 11 fish in the sample. This information is in the form of bounding boxes.

4.2 Qualitative Results on Main Video Samples

What is evident from observing the output video of both trackers is that they both perform poorly in terms of maintaining an accurate count of the fish, as well as tracking individual fish for several frames.

4.2.1 ByteTrack

True positives

A true positive occurs in the frame when the fish has been correctly identified as a fish with a bounding box.

True positives are most common when the zebrafish are perpendicular to the camera, and especially when they are in the center portion of the fish tank. This holds true for all video samples regardless of frame rate, resolution, or lighting. It seems like the most crucial criterion for detection to be made is that the characteristic zebra stripes are visible. See figure 4.1 for typical examples of true positives. All images shown are from the first frame that the fish is identified.

While it is more unlikely, it is also possible for true positives to occur when the fish are more towards the edges of the fish tank, or when they are turned towards or away from the camera. There are, for example, instances where a fish is detected when facing nearly exactly towards the camera. See figure 4.2 for examples. Once again, all the examples shown are the first frame in which the fish is identified.

False positives

False positives occur when it is predicted that a fish will be where there is, in fact, no fish.

False positives are most common towards the edges of the fish tank. In all observed cases, false positives occur because the reflection of an actual fish is predicted to be a fish by the model. These reflections are observed to occur on the bottom of the water surface, or on the sides of the fish tank. See figure 4.3 for examples of false positives.

False negatives

A false negative is when the model predicts that a real fish is not a fish. Unfortunately, because of the poor performance of the model, there are many instances of false negatives. The most common ones are when the fish is towards the edges of the tank, or facing towards or away from the camera. Collectively, the fish are more likely to be in the right-most part of the tank, and the top-right corner in particular. This is most likely due to

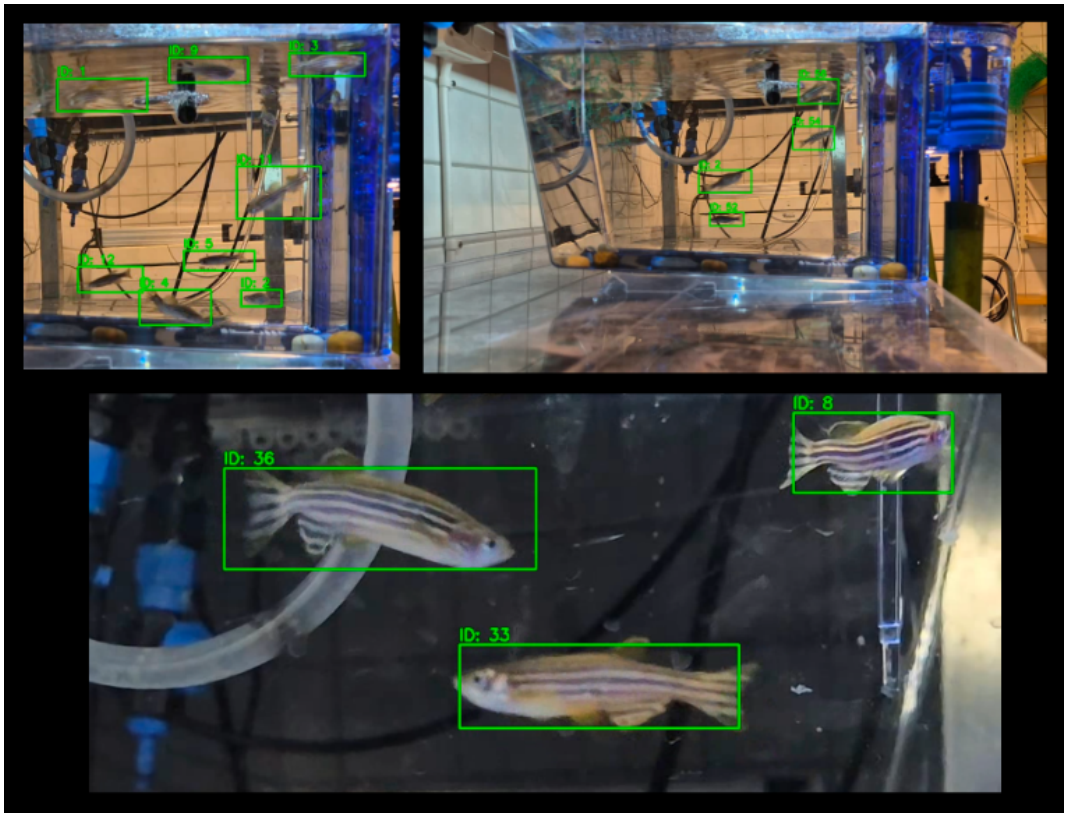


Figure 4.1: Typical ByteTrack true positives.

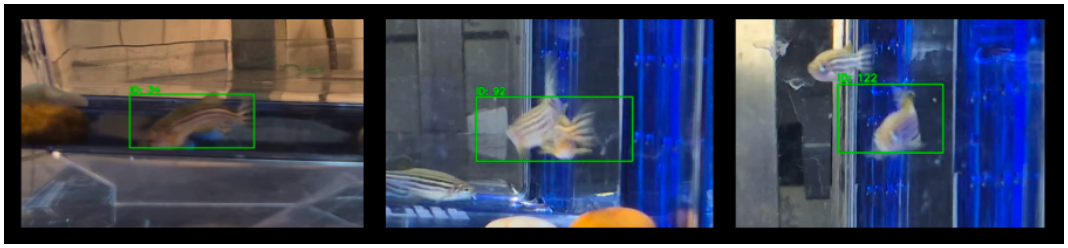


Figure 4.2: Edge case ByteTrack true positives.

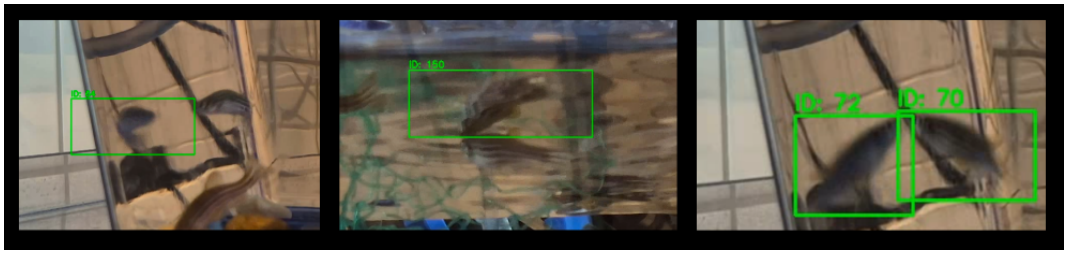


Figure 4.3: ByteTrack False positives. The first and third image show false positives from wall reflections. The middle image show a false positive from the water surface.

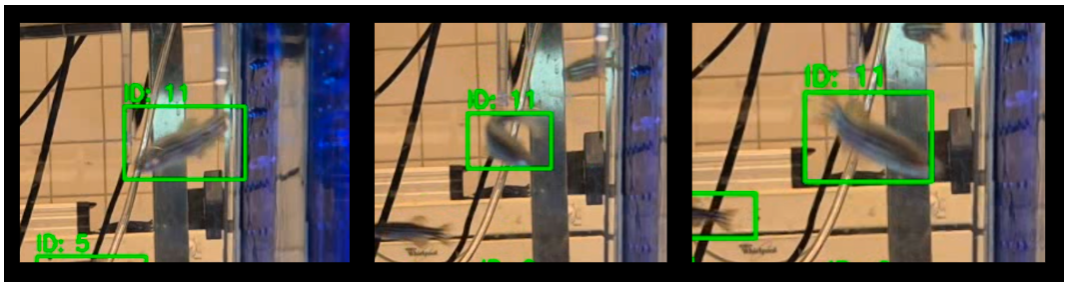


Figure 4.4: ByteTrack successfully maintains track and ID during the full U-turn sequence.

them being fed in this corner, and it causes a lot of occlusions where false negatives are likely.

There are also instances of false negatives where the fish is in the center of the tank and perpendicular to the camera. In most cases, these circumstances will result in a true positive, but there are still examples of false negatives that occur under these circumstances.

Lastly, there are also instances where the tracker has a track on a fish and then loses track of the fish despite the conditions for identification being good. These occurrences are further discussed in the section about Permanent loss of ID or incorrect ID.

Sequences of maintained ID

When talking about maintaining ID, this can essentially mean two scenarios. The first scenario is when the tracker maintains the correct and uninterrupted track, and thus also ID, of the fish for several frames. The second scenario is sequences where the tracker has tracked a fish, loses it, and then manages to successfully reidentify the fish.

The most common instances of maintained ID under the first scenario are simply when the fish are swimming perpendicular to the camera, typically in constant speed and direction. Under these circumstances, ByteTrack is often able to maintain a correct track and ID until the fish turns, is occluded, or changes speed. Fish that are very still are also often tracked if they show their zebra stripes while still. Although it is uncommon, there have also been observed instances in which the correct track and ID have been maintained even though the fish has made a full U-turn. Figure 4.4 depicts this example. Typically, for this to happen, the fish need to swim slowly, as the rate of change in speed and direction plays a major role in tracker performance.

When it comes to the second scenario, these instances are typically observed when fish are being occluded, or changing speed or direction abruptly. Under these circumstances, successful reidentification can absolutely occur, but it very much seems like a 50-50 chance. In figure 4.7 a good example of a successful reidentification of a fish changing direction completely is shown.

Fish may also occlude each other, and in these instances the reidentifiaction may be successful. For the best likelihood of reidentification, the fish should preferably be swimming in as opposite directions as possible, and maintain constant speeds.

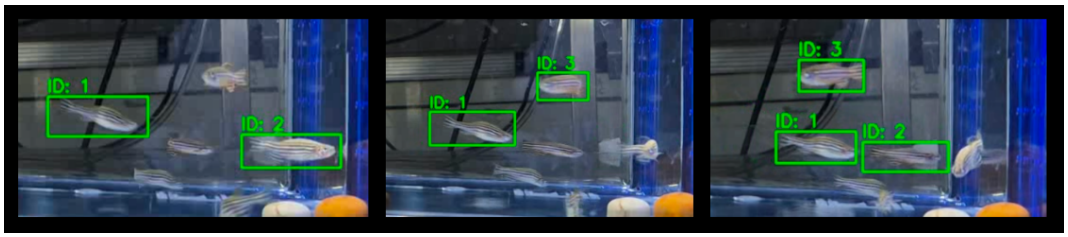


Figure 4.5: ByteTrack loses track of ID 2 and incorrectly reassigns ID 2 to the fish coming from behind.

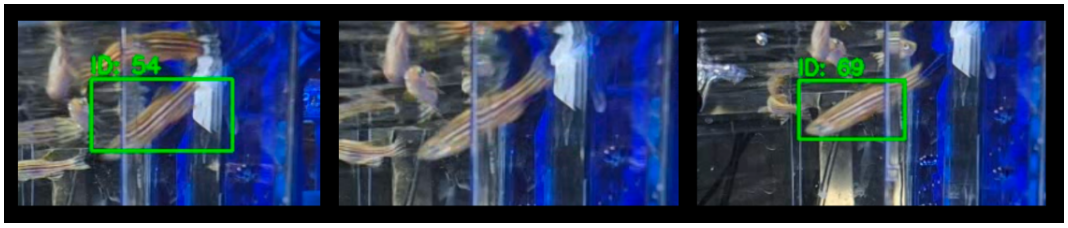


Figure 4.6: A tracked fish is briefly lost and shortly after identified as a new fish.

Permanent loss of ID or incorrect ID

This section covers observations where the tracker loses a fish and never reidentifies it again, as well as when the tracker thinks it has reidentified the fish, but has, in fact, assigned the ID to another fish.

As mentioned in the previous section, it seems like a 50-50 chance that a fish is correctly reidentified when the fish changes speed, direction, or is occluded. Figure 4.5 shows an example in which a tracked fish changes direction, is lost, and the tracker then assigns the ID to another nearby fish. A prerequisite for an ID to be assigned to a different fish is that the other fish has to be in the same general area as the initially tracked fish.

Another example is shown in figure 4.6 where a fish is swimming in a general direction, is lost for just a few frames, and then identified as a new fish, even though it is still swimming in the same direction and with the same speed.

When it comes to fish occluding each other, it is likely that identities are not maintained, at least for both of them. If they are going in the same direction, or if they change direction or speed during occlusion, the reidentification is less reliable. There are also occurrences

Figure 4.7: A successful ByteTrack reidentification. ID 92 is swimming left, changing direction and lost by the tracker, and lastly swimming right and reidentified by the tracker.

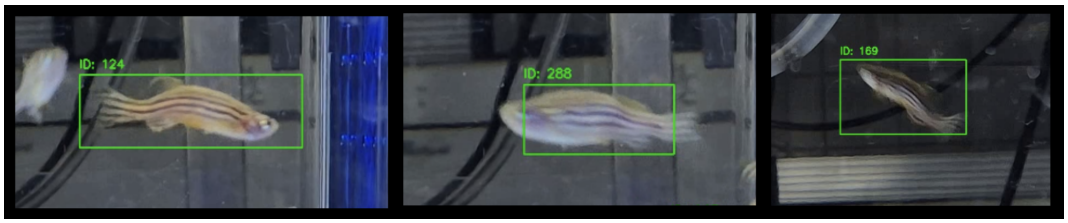


Figure 4.8: DeepSORT true positive examples

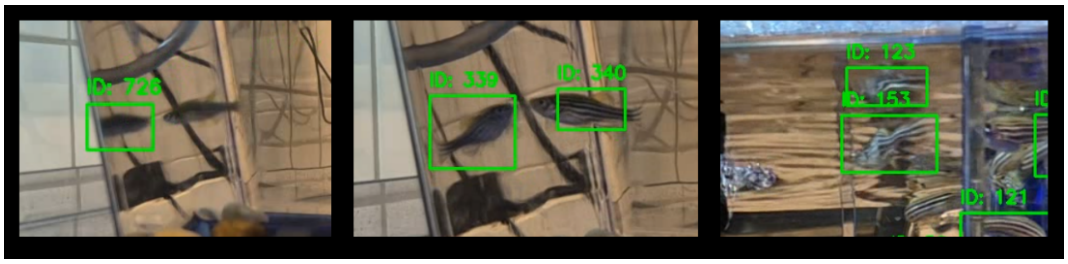


Figure 4.9: The only observed DeepSORT false positives. The first and second images show false positives in the wall reflection. The third image show a false positive in the water surface reflection.

where the wrong ID is assigned to the wrong fish after being lost. This can, for example, happen if two fish going in the same general direction occlude each other. A common sequence is that the ID of the first fish is lost, and that the second fish' ID is assigned to the first fish.

4.2.2 DeepSORT

True positives

Unlike ByteTrack, DeepSORT is more restrictive with flagging true positives. With DeepSORT, true positives generally only occur when the fish is perpendicular to the camera, and true positives are more likely toward the center of the tank. Unlike ByteTrack, with DeepSORT it is quite unlikely that a fish will be correctly identified if it is facing towards or away from the camera. The more restrictive detection is due to DeepSORT's built-in logic that tracking can only begin if an object has been detected for five consecutive frames.

All of the image examples shown in figure 4.8 are from the first frame that the fish has been identified, and it is quite clear that all the true positive examples are of the same characteristics.

False positives

When it comes to DeepSORT, false positives are extremely rare. With ByteTrack, false positives could occur in the reflections of the walls of the fish tank as well as on the surface of the water. When it comes to DeepSORT, there were only two instances of fish that were incorrectly identified in the wall reflection of the tanks, and only one instance of fish being incorrectly identified on the water surface. See figure 4.9 for these.

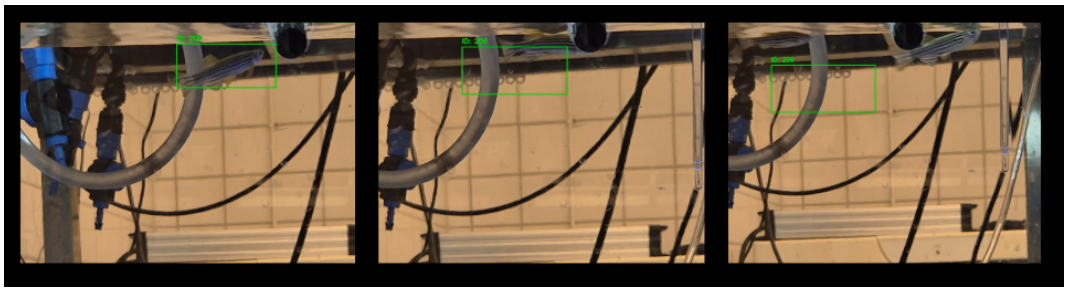


Figure 4.10: The tracker loses the fish but lets the bounding box drift for 30 frames before deleting it.

However, it should be mentioned that false positives will occur very frequently after an initial true positive detection of a fish. If DeepSORT loses a detected fish, the bounding box will continue in the direction in which the fish was last known to have traveled. If the fish is successfully reidentified, the bounding box will reattach to the fish. If the fish is not reidentified, the bounding box will drift for 30 frames before being deleted. Technically, all of these frames count as false positives. See figure 4.10 for an example of such a drifting bounding box.

False negatives

DeepSORT produces more false negatives than ByteTrack. As mentioned in the section covering True positives, DeepSORT is more restrictive with flagging true positives, and therefore false negatives will also be more common. Most commonly, false negatives occur when the fish are towards the edges of the tank, or when they are facing towards or away from the camera. However, false negatives are still quite common when the fish is perpendicular to the camera and towards the center of the tank.

Just as with ByteTrack, DeepSORT also struggles with false positives occurring in the rightmost part of the tank, where the fish are likely to gather due to feeding habits.

Sequences of maintained ID

When it comes to correctly tracking fish, DeepSORT performs fairly poorly. Most of the time, a successful reidentification happens only if the fish is traveling in almost the same speed and direction as when it was lost. Because the tracker easily loses track of a fish for only a few frames at a time, this cycle of fish being lost and reidentified can happen several times while a fish swims in a constant speed perpendicular to the camera. While it is rare, there are observed instances of the correct fish being reidentified if it has been lost due to a sudden change of direction. Figure 4.11 depicts one such occurrence. There are also few instances of the correct fish being reidentified after swimming in front of each other (see figure 4.12 for an example).

Permanent loss of ID or incorrect ID

Unsuccessful attempts to reidentify the fish can essentially happen at all times once a fish has been lost. Most often, unsuccessful reidentification occurs when the fish changes

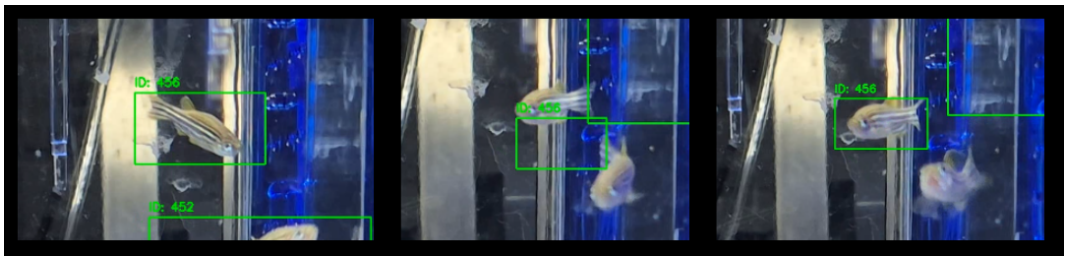


Figure 4.11: DeepSORT successfully reidentifies a fish that has abruptly changed direction.

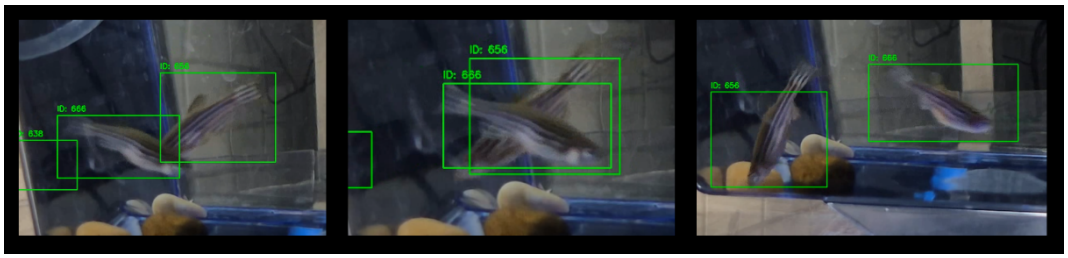


Figure 4.12: DeepSORT correctly maintains the ID of two fish crossing paths.

direction or speed drastically. In figure 4.13, a fish can be seen changing direction, which causes the tracker to lose the fish, and then assigns the existing ID to a new fish while giving the first fish a new ID. Another example can be seen in figure 4.5, where a fish swimming in the same direction and speed is briefly lost and then given a new ID while the old ID is still alive and its corresponding bounding box is still near the fish.

Just as with ByteTrack, if the fish are being occluded by each other, there is also a great risk that either one or both IDs are lost or wrongly assigned. This appears to be more common in DeepSORT than with ByteTrack. However, there are also instances where a fish is lost while traveling at a more or less constant speed and direction, and then is not properly reidentified, seemingly without reason. This is a pattern that was essentially only seen with DeepSORT.

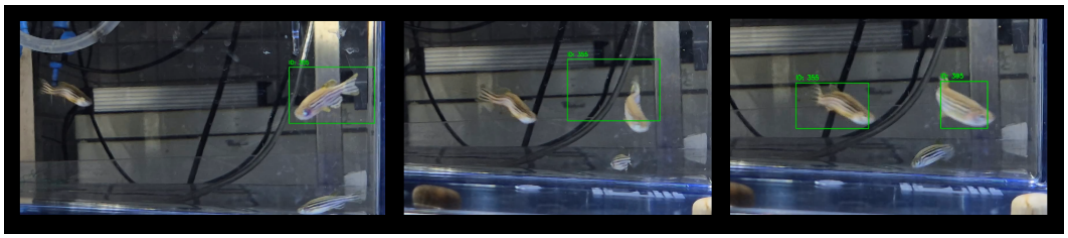


Figure 4.13: ID 355 changes direction, which causes DeepSORT to lose the fish. ID 355 is then reassigned to a different fish while the actual ID 355 gets a new ID.

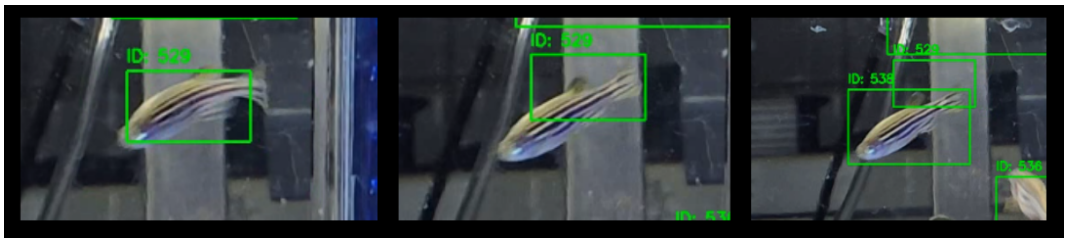


Figure 4.14: ID 529 is swimming in the the same direction and speed, but is still lost by DeepSORT, and then assigned ID 538 while the previous bounding box is near the fish.

4.3 Online Quantitative Results on Main Video Samples

4.3.1 Overview Summary

In this section, a high-level snapshot of each tracker's aggregated performance across all 12 clips within the main video samples will be presented.

In Table 4.2, the overall averages for the 12 video samples are presented for each tracker. The meaning of each metric is presented below.

- *FPS* - The average rate at which frames are processed each second.
- *Unique Fish* - The total number of unique fish identified for each video clip.
- *Unique Tracks* - The total number of unique tracks identified for each video clip. Each unique fish can have several associated tracks if the fish is lost and reidentified.
- *Frames/Fish* - The average number of frames for which each unique fish is tracked.
- *Frames/Track* - The average number of frames for which each unique track exists.
- *Tracks/Fish* - The average number of tracks associated with each identified fish.
- *ReID Fish* - The total number of instances where a lost fish is reidentified.
- *ReID succ. rate* - The share of lost fish instances where the fish is successfully reidentified.

Table 4.2: Average online metrics for ByteTrack and DeepSORT

Tracker	FPS	Unique Fish	Unique Tracks	Frames/ Fish	Frames/ Track	Tracks/ Fish	ReID Fish	ReID succ rate
ByteTrack	11.43	149.42	711.92	26.25	5.37	4.77	556.50	77.99%
DeepSORT	8.66	197.5	398.33	12.57	6.17	1.98	202.75	48.81%

To further highlight the differences between the performance of the trackers, three bar charts are included for better visualization. In the first chart, depicted in figure 4.15, the metrics FPS, Frames/Fish, Frames/Track, and Tracks/Fish are compared. What is evident is that ByteTrack runs faster, is able to track each fish for more frames, and is also able to track each fish over more separate tracks. The only metric where DeepSORT excels, albeit by a small margin, is the average number of frames for which each track is maintained.

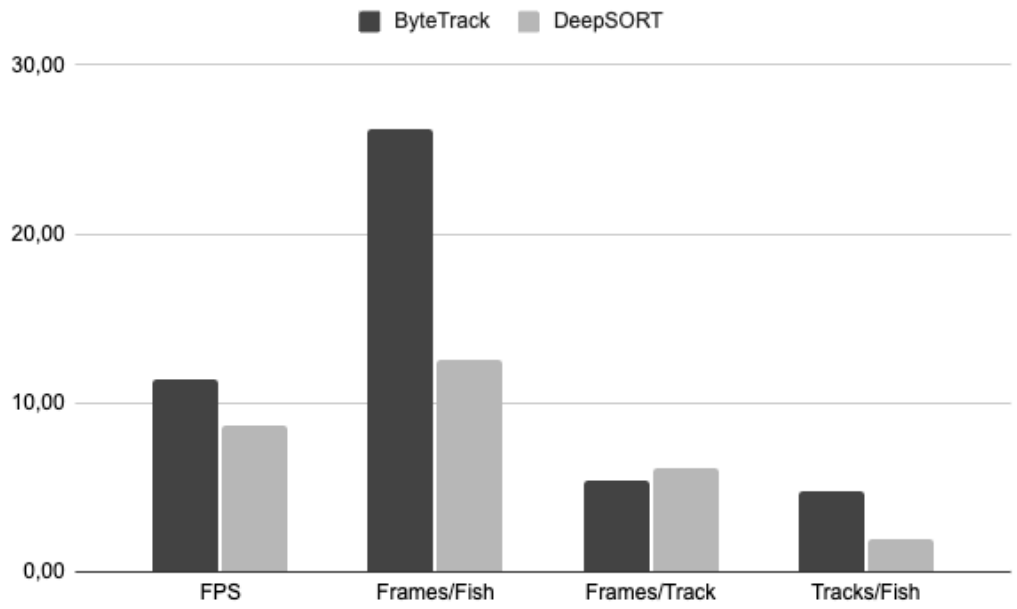


Figure 4.15: Bar chart comparing ByteTrack and DeepSORT on FPS, Frames/Fish, Frames/Track, and Tracks/Fish.

The next bar chart is shown in figure 4.16 where the metrics Unique fish, Unique tracks, and ReID fish are compared. The metric Unique fish should preferably be as close to 11 as possible, so while ByteTrack produces a slightly better value than DeepSORT, both of the trackers perform poorly. In the other two metrics, ByteTrack produces significantly more hits than DeepSORT. Having more unique tracks could be a sign of a tracker that is unable to maintain fish ID, but in this case we can also see that ByteTrack significantly outperforms DeepSORT in the reidentification of fish. In figure 4.17 this relationship is further illustrated where it is shown that ByteTrack is significantly more likely to reidentify a lost fish.

4.3.2 Tracker Comparison Under Matched Conditions

In this section, both trackers will be compared when running the same exact video clip. To help visualize how the trackers performed compared to each other, figure 4.18 will be a central tool.

Figure 4.18 shows the tracker performance in pairs for each video configuration. So for the first pair, ByteTrack and DeepSORT are compared on the 30 FPS, 720p resolution video in dark lighting. The color-graded row under the pair shows the difference between the metrics of the two trackers. For all columns other than Unique fish and Unique tracks, a greater number is beneficial to ByteTrack. For Unique fish and Unique tracks, a greater number is beneficial to DeepSORT. The color grading is based on the per-column range of differences values. This means that for the FPS column, a higher (blue) value will indicate that ByteTrack performed better than in other video samples, and a lower (yellow) value indicates that DeepSORT performed better than in other video samples. This means that a yellow cell will not necessarily imply that DeepSORT performed better than ByteTrack for that video sample and metric, but rather that compared to all differences for that metric, it performed the best on this specific video sample.

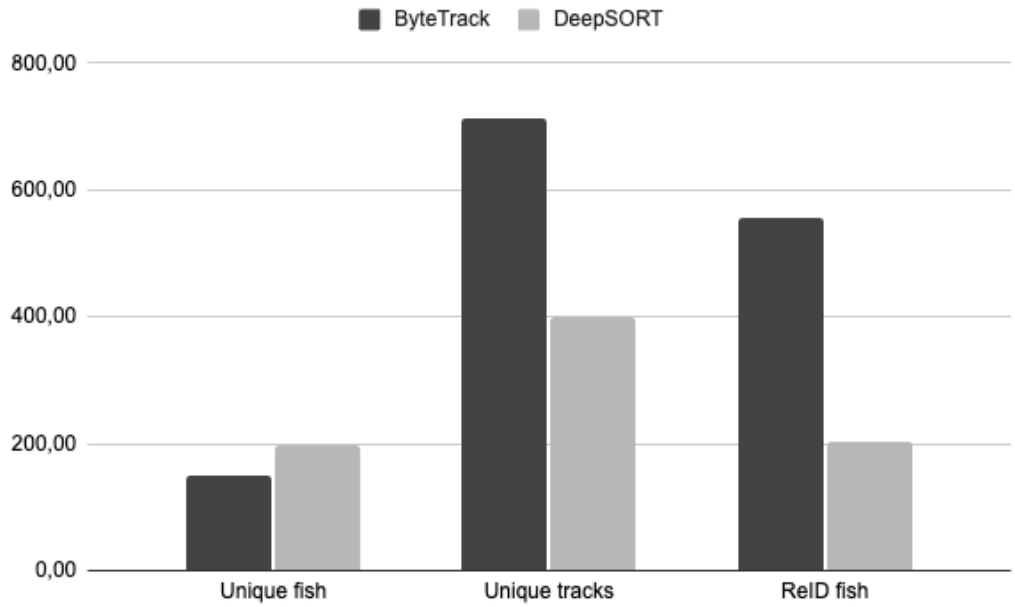


Figure 4.16: Bar chart comparing ByteTrack and DeepSORT on Unique fish, Unique tracks, and number of reidentified fish.

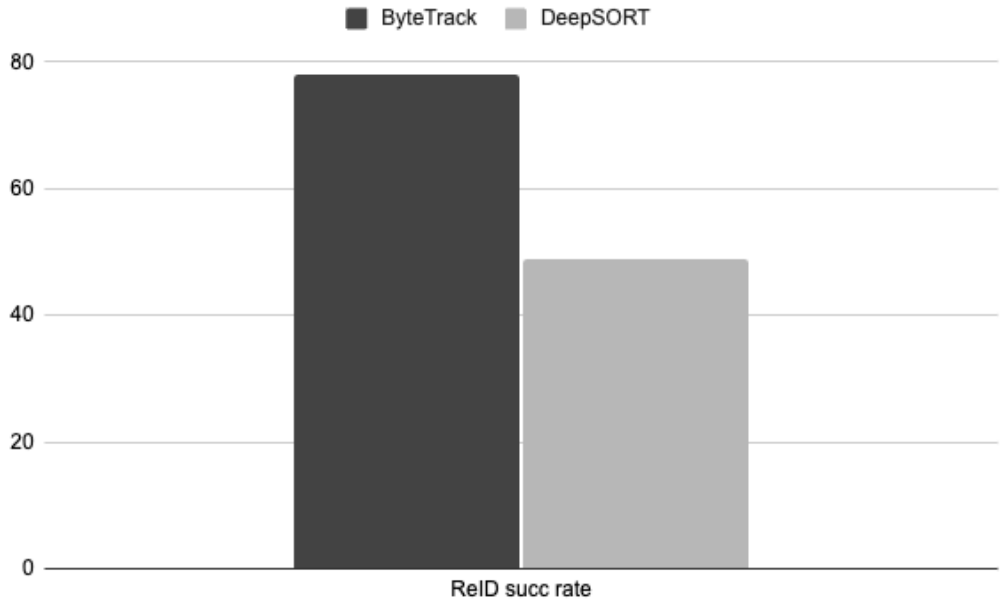


Figure 4.17: Bar chart comparing ByteTrack and DeepSORT on their reidentification success rates.

Model	FPS	Res	L/D	FPS	Unique fish	Unique tracks	Frames/Fish	Frames/Track	Tracks/Fish	ReID fish	ReID succ rate
ByteTrack	30	720	D	13,15	134	591	18,12	4,11	4,41	457	77,72%
DeepSORT	30	720	D	9,97	158	266	7,9	4,66	1,68	110	41,34%
				3,18	**−24**	**325**	**10,22**	**−0,55**	**2,73**	**347**	**36,38%**
ByteTrack	30	720	L	13,13	159	699	24,63	5,6	4,4	540	77,70%
DeepSORT	30	720	L	9,37	207	417	12,42	6,03	2,01	213	51,08%
				3,76	**−48**	**282**	**12,21**	**−0,43**	**2,39**	**327**	**26,62%**
ByteTrack	30	1080	D	12,82	147	650	18,97	4,29	4,42	500	77,16%
DeepSORT	30	1080	D	9,65	164	313	9,36	4,9	1,91	149	47,60%
				3,17	**−17**	**337**	**9,61**	**−0,61**	**2,51**	**351**	**29,56%**
ByteTrack	30	1080	L	12,48	154	722	19,71	4,2	4,69	568	79,11%
DeepSORT	30	1080	L	9,57	186	337	8,87	4,85	1,81	154	45,70%
				2,91	**−32**	**385**	**10,84**	**−0,65**	**2,88**	**414**	**33,41%**
ByteTrack	30	4K	D	11,1	127	501	20,89	5,3	3,94	373	74,75%
DeepSORT	30	4K	D	8,62	160	267	10,42	6,22	1,67	109	40,82%
				2,48	**−33**	**234**	**10,47**	**−0,92**	**2,27**	**264**	**33,93%**
ByteTrack	30	4K	L	11,03	144	614	17,9	4,2	4,26	469	76,63%
DeepSORT	30	4K	L	8,53	148	269	9,33	5,13	1,82	121	44,98%
				2,5	**−4**	**345**	**8,57**	**−0,93**	**2,44**	**348**	**31,65%**
ByteTrack	30	8K	D	7,59	152	526	17,76	5,13	3,46	374	71,24%
DeepSORT	30	8K	D	6,24	158	277	10,4	5,93	1,75	119	42,96%
				1,35	**−6**	**249**	**7,36**	**−0,8**	**1,71**	**255**	**28,28%**
ByteTrack	30	8K	L	7,64	178	721	20,65	5,1	4,05	543	75,63%
DeepSORT	30	8K	L	6,08	223	377	9,76	5,74	1,69	155	41,11%
				1,56	**−45**	**344**	**10,89**	**−0,64**	**2,36**	**388**	**34,52%**
ByteTrack	60	1080	D	12,88	143	856	40,98	6,85	5,99	694	81,55%
DeepSORT	60	1080	D	9,68	219	576	20,21	7,56	2,63	360	62,50%
				3,2	**−76**	**280**	**20,77**	**−0,71**	**3,36**	**334**	**19,05%**
ByteTrack	60	1080	L	12,91	160	888	31,81	5,73	5,55	714	80,95%
DeepSORT	60	1080	L	9,18	244	530	15,15	6,85	2,17	291	54,91%
				3,73	**−84**	**358**	**16,66**	**−1,12**	**3,38**	**423**	**26,04%**
ByteTrack	60	4K	D	11,15	142	877	41,12	6,66	6,18	719	82,27%
DeepSORT	60	4K	D	8,66	240	609	18,44	7,21	2,54	372	61,08%
				2,49	**−98**	**268**	**22,68**	**−0,55**	**3,64**	**347**	**21,19%**
ByteTrack	60	4K	L	11,25	153	898	42,47	7,24	5,87	727	81,14%
DeepSORT	60	4K	L	8,33	263	542	18,62	8,98	2,06	280	51,66%
				2,92	**−110**	**356**	**23,85**	**−1,74**	**3,81**	**447**	**29,48%**

Figure 4.18: A heatmap showing under which circumstances each tracker performed comparatively better or worse against the other tracker. Blue signals ByteTrack advantage, and yellow signals DeepSORT advantage.

Looking at figure 4.18, there are several interesting indications to pay attention to. The initial general pattern that can be identified is that for nearly all metrics, ByteTrack performed comparatively better on the 60 FPS video samples. The obvious outlier would be the ReID success rate, where DeepSORT performed comparatively better on the higher frame rate. Furthermore, it is quite obvious that ByteTrack performs comparatively better in terms of FPS the lower the resolution is, which holds true both for the 30 FPS and 60 FPS video categories. Another interesting pattern that mostly holds true is that when it comes to Unique tracks, ByteTrack will in most cases perform the best when lights are on, and conversely DeepSORT will perform better in dark videos. Lastly, an interesting thing to note is that DeepSORT performed comparatively well for nearly all metrics on the 30 FPS, 8K, dark video sample.

However, the single most important point to make from observing figure 4.18 must still be the absolute values shown in each metric difference cell. ByteTrack performs better than DeepSORT in nearly all metrics and video samples. The only metrics where DeepSORT performs better are all measurements of Unique tracks, as well as all measurements of Frames/Track. Essentially, DeepSORT creates fewer tracks and keeps each track alive longer than ByteTrack.

4.4 MOT-Challenge Results on Ground Truth Sample

In this section, a quantitative comparison of the two object trackers is presented using a ground-truth video sample. The ground truth sample used is a 10 second, 30 FPS video of 11 zebrafish. By comparing the trackers against the ground truth sample, they can be evaluated using the standard MOTChallenge metrics. These metrics will reveal how the trackers perform in terms of detection, association, overall accuracy, and track stability. All key metrics from the MOTChallenge are summarized in table 4.3.

4.4.1 Overview of Metrics

Three different groups of metrics are reported:

- **Overall accuracy**: HOTA (Higher Order Tracking Accuracy), MOTA (Multiple Object Tracking Accuracy), and IDF1 (identity F1).
- **Detection performance**: DetRe (detection recall), DetPr (detection precision), and MOTP (localization precision).
- **Association performance**: AssRe (association recall), AssPr (association precision), and IDSW (number of ID switches).

Details about how each metric is calculated and what information they reveal will be included in each of their dedicated sections below.

4.4.2 Detection Performance

Detection performance is characterized by DetRe, DetPr, and MOTP. Their equations are shown below.

$$\text{DetRe} = \frac{\text{TP}}{\text{TP} + \text{FN}} \times 100\% \tag{4.1}$$

Detection recall simply measures the share of total ground-truth objects that were detected.

$$\text{DetPr} = \frac{\text{TP}}{\text{TP} + \text{FP}} \times 100\% \tag{4.2}$$

Detection precision, on the other hand, measures the share of detections that were correct.

The MOTP metric measures the average dissimilarity between the true positive predictions and the ground truth bounding boxes. See equation 2.2 for how it is calculated.

- **DetRe**: ByteTrack recalls 16,48% of ground-truth boxes, while DeepSORT recalls 20,02%.

- **DetPr**: ByteTrack is fairly precise (70,81%) compared to DeepSORT (29,18%), indicating far fewer false alarms.
- **MOTP**: Both trackers localize detected fish with similar tightness (ByteTrack 71.97%, DeepSORT 69.57%).

ByteTrack's detector sacrifices recall to achieve higher precision, whereas DeepSORT retrieves a comparable fraction of fish at the cost of many spurious detections.

4.4.3 Association Performance

Association metrics reveal how well the trackers maintain consistent identities. An association in this context is simply the tracker maintaining the same ID for the same ground-truth object between frames. The relevant equations and their meanings are covered in the following.

$$\text{AssRe} = \frac{\text{TPA}}{\text{TPA} + \text{FNA}} \times 100\% \tag{4.3}$$

Association recall measures the fraction of correctly detected objects whose identities the tracker maintained. False Negative Association (FNA) represents an instance in which the ID is switched for the same fish.

$$\text{AssPr} = \frac{\text{TPA}}{\text{TPA} + \text{FPA}} \times 100\% \tag{4.4}$$

Association precision measures the fraction of all identity association attempts that were correct. False Positive Associations (FPA) represents instances where the ID of one fish is reassigned to another fish.

IDSW (ID switches) simply measures the number of times that the tracker switches ID, so there is no equation to provide. The closer to 0 the IDSW metric is, the better.

IDF1 measures the harmonic mean of identity precision and recall, summarizing how well the tracker keeps consistent IDs over time. See equation 2.3 for how IDF1 is calculated.

- **AssRe**: ByteTrack correctly reassociates 4,69% of true positives, DeepSORT 7,70%.
- **AssPr**: ByteTrack's association precision is 74,02%, versus 27,56% for DeepSORT.
- **IDSW**: ByteTrack committed 101 ID switches. DeepSORT committed 62.
- **IDF1**: ByteTrack achieves 11,18%, DeepSORT 9,57%.

Both trackers struggle to maintain consistent IDs (low IDF1), but ByteTrack is more conservative, making more switches overall but yielding higher precision when it does associate.

4.4.4 Overall Accuracy

Overall tracker accuracy is captured by HOTA and MOTA.

HOTA is a single metric that balances the detection and association qualities of the tracker into one single metric. See equation 2.6 for how HOTA is calculated. MOTA is a metric that measures the overall accuracy of the tracker by penalizing all errors made. Errors include false positives, false negatives, and ID switches, and the sum of these is normalized by the total number of ground-truth boxes. See equation 2.1 for how MOTA is calculated. For both of these metrics, a higher value is preferred.

- **HOTA**: ByteTrack scores 8,62%, DeepSORT 9,31%.
- **MOTA**: ByteTrack achieves 17,37%, whereas DeepSORT is negative at –27,99%.

ByteTrack achieves a positive MOTA of 17,37%, indicating that it makes substantially fewer combined errors (FP + FN + IDSW) than there are ground-truth instances. In contrast, DeepSORT records a MOTA of –27,99 %, which means that its total error count exceeds the total number of ground-truth instances.

DeepSORT's negative MOTA reflects its high combined FP, FN, and IDSW count. Although DeepSORT slightly exceeds ByteTrack on HOTA, its overall error rate is still substantially larger.

Table 4.3: Comparison of primary MOTChallenge metrics between ByteTrack and DeepSORT.

Category	Metric	ByteTrack	DeepSORT	Description
Overall	HOTA	8.62 %	9.31 %	Balances detection and association quality.
	MOTA	17.37 %	–27.99 %	Classic accuracy (errors normalized by total GT).
Detection	DetRe	16.48 %	20.02 %	Fraction of GT objects successfully detected.
	DetPr	70.81 %	29.18 %	Fraction of detections that are true objects.
	MOTP	71.97 %	69.57 %	Tightness of the bounding-box fits.
Association	AssRe	4.69 %	7.70 %	Fraction of TP detections whose ID was kept.
	AssPr	74.02 %	27.56 %	Fraction of ID associations that were correct.
	IDSW	101	62	The total number of identity swaps.
	IDF1	11.18 %	9.57 %	How well identities are maintained.

4.5 Discussion

In this section, a discussion around the obtained results will be presented in two parts. First, the overall performance differences between the trackers will be discussed. Secondly, a discussion will be held on how well these trackers are applicable to the problem at hand.

4.5.1 ByteTrack vs. DeepSORT

While both of these trackers have many similarities in their algorithms, they also share a few crucial differences whose performance impacts become interesting to analyze with all results at hand. To quickly repeat and remind, both trackers employ Kalman filters for track predictions and the Hungarian algorithm for detection and prediction matching. ByteTrack tries to match all detections in two passes, one for high-confidence detections and one for low-confidence detections, whereas DeepSORT only tries to match high-confidence detections. DeepSORT, on the other hand, factors in both motion and appearance features in its prediction modeling, while ByteTrack relies only on motion.

By first analyzing the online metrics, it is evident that ByteTrack is consistently running at a higher frame rate than DeepSORT. On average, it runs roughly 3 frames per second faster across all video samples. Furthermore, ByteTrack is able to track each fish for more than double the amount of frames than DeepSORT is able to, albeit with more fragmented tracks. Importantly, ByteTrack is able to identify fewer unique fish, which is in large part due to its greater ability to reidentify lost fish.

Looking at the MOTC metrics, the story is in large part the same. When it comes to HOTA, DeepSORT is marginally better than ByteTrack, driven by its slightly better association recall and fewer ID switches, but in all other metrics, ByteTrack performs better. While ByteTrack's advantage in MOTP and IDF1 are fairly small, the difference in the MOTA metric is substantial and indicates a large discrepancy in how many more errors that DeepSORT commits.

When factoring in the qualitative analysis, many of the indications seen in the qualitative metrics are confirmed. A natural first aspect to ascertain is that both trackers behave much in the same way. Both struggle with aspects such as maintaining ID for longer sequences and identifying fish that are not perfectly visible in the middle of the tank. However, what is evident from observing the video samples is that ByteTrack simply handles most of the tracking aspects in a higher-quality fashion. It is apparent that it is better than DeepSORT in identifying the fish, tracking the fish, and handling occlusions. Simply put, while both trackers make the same sorts of mistakes, ByteTrack is just less likely to do them.

Taking all this together, it is clear that ByteTrack's methodology of trying to match all detections in two steps while keeping the rest of the model lightweight gives it an advantage in terms of speed and precision. DeepSORT's appearance feature modeling seems to offer near-zero benefit which in all certainty is because of the high degree of similarity between the fish.

4.5.2 Applicability for Zebrafish Tracking

The previous section made it rather clear that while both trackers are lacking in key object tracking metrics, ByteTrack is in most aspects performing better out of the two. However, when putting these results into the context of the results that are evaluated in this thesis, it is quite clear that none of the trackers will be applicable in their current form. Recall that the MTM lab is looking for a solution that can keep an accurate count of the zebrafish in the tank, track individual zebrafish continuously, and can do this at real-time speeds.

Looking at these requirements one by one, we can see that for maintaining an accurate count of the fish, the best possible score a tracker could achieve is 11 unique fish. ByteTrack counted an average of 149,4 unique fish and DeepSORT counted an average of 197,5

unique fish. When it comes to tracking individual fish, a perfect score would have been an average of 1200 frames per fish. ByteTrack managed 26,25 frames per fish, and DeepSORT averaged 12,57 frames per fish. Lastly, when it comes to frame rates, the best possible average score would have been 40. ByteTrack averaged 11,43 and DeepSORT scored 8,66.

These perfect scores are clearly unattainable even for state-of-the-art trackers on linearly moving objects such as cars or pedestrians, but they do highlight that these results are very far off from a viable, permanent solution for the MTM lab. While there are suggestions of future work presented in the next chapter, it is still evident that new algorithms that can better model the nonlinear fashion of the zebrafish motion most likely must be attained before an accurate live-environment deployment can be made.

Chapter 5
Conclusions

5.1 Review of Project Goals

As stated in section 1.1, the overall goal of this project was to lay the foundations for a fully computerized future implementation of zebrafish monitoring in the laboratory setting at MTM. In order to achieve this goal, a number of subproblems were defined, namely to identify an object detector suitable for detecting zebrafish, to identify suitable object trackers for the project and to implement them, and finally to evaluate the object trackers on relevant metrics to assess their current abilities of tracking zebrafish and what is needed to improve them.

In regards to the first of the subproblems, bringing the problem to a conclusion was luckily quite trivial. The existing detector implementation by Storm (2024)[6] was readily available. It was first tested using still images of zebrafish in the MTM laboratory setting to ensure that it worked. Once that had been established, work could commence on implementing the zebrafish detector in the object tracking models, which fortunately was quite trivial except for the bounding-box conversion logic between the detector and tracker.

Identifying suitable object trackers to evaluate in this project was a much more time-consuming task than expected. The process involved analyzing a plethora of research papers introducing various object tracking models and sifting through the information to deduce which ones would be most suitable for the specific task at hand. In the end, the two object trackers that were selected were so due to both of them showcasing strong accuracy while maintaining good processing speed. The aspect of processing speed was important because the ambition is that a future implementation should be able to track the zebrafish in real time. The object trackers were also selected because they had different association logic. ByteTrack's association logic relies only on motion, while DeepSORT combines motion and appearance features in the association logic. The motivation here was that it would be interesting to evaluate whether appearance aspects would affect the results.

The performance evaluation of the two trackers is probably where this project falls a bit short, mainly due to time constraints. The two trackers were evaluated only on a small sample of zebrafish videos, so it was not possible to evaluate if the trackers performed better for certain video properties or other, or to draw any other decisive conclusions. Furthermore, the ground-truth video sample was very short at only 10 seconds, which affected the inference that could be drawn from the results on the MOTChallenge metrics. If this project had been for the duration of perhaps two extra weeks, more video material

could have been obtained, and more material could have been labeled with ground-truth bounding boxes, which most likely would have aided in producing higher-quality metrics that more analyses could have been drawn from. Furthermore, due to the limited metrics that could be obtained, it was decided to complement the quantitative analysis with a qualitative analysis. However, my own perception is that that analysis gave fairly similar answers for both trackers and did not provide much added value in terms of tracker properties.

Despite the shortcomings with the performance evaluation, it should still be stated that important conclusions can be drawn from the performance evaluation with regard to the feasibility of implementing the trackers in the given laboratory scenario, as well as which tracker is better suited than the other. Overall, I am very happy with the way the project goals were met.

5.2 Context

When it comes to the context i in which this thesis exists, there are several areas that are relevant. The obvious initial aspect to take into account is the welfare of the zebrafish themselves, but there are also scientific and societal considerations that are important to take into account.

The maintenance of zebrafish in a laboratory environment comes with a number of considerations to keep in mind. Since 5 July 2022, zebrafish are a listed species according to the EU animal health regulation [34]. Aspects such as the number of fish per liter, the conditions of the water, and the visual surroundings are all things to take into account. In the context of this thesis, the main priority has been to avoid causing any stress on the fish. This has been done by minimizing the number of visits to the laboratory, and that while there, not make any sudden noises or movements that could startle the fish. The camera was always placed well away from the tank, and no lights, alerts, or other distractions were enabled on the camera. In addition to the physical visits to the lab and the video recordings, no other elements of this thesis had any impact on the fish.

In a potential future, permanent implementation of a multi-object tracker in the laboratory, naturally considerations will have to be taken in regards to how the camera is mounted and what impact that will have on the fish. Another important aspect to keep in mind is that while a permanent implementation would reduce the need for manual labor by the lab personnel, if the tracker has systemic errors, such as incorrect counting or ID tracking, unhealthy fish could remain unchecked for too long, or the staff may need to intervene multiple times, causing stress for the fish. Therefore, careful validation and testing of a potential final tracking model will be required.

Another important consideration is the scientific sphere in which this thesis will exist. The ability of peers to reproduce and validate the results of this thesis lies at the root of scientific integrity, and as such, data sets, parameter settings, and evaluation metrics must be reported transparently. By open-sourcing code and publishing the ground-truth annotations, the reproducibility is upheld, and peers are enabled to verify or build on top of the results found. This is especially important if the long-term goal of the MTM lab is to advocate for the further development of a permanent zebrafish tracking solution.

Lastly, from a purely societal perspective, it is important to highlight that there are some clear benefits to accurate tracking of zebrafish that motivate why continued work in this field is needed. The reduction in manual labor from lab staff will mean that they can shift

their efforts from non-fulfilling manual tasks to more productive research contributions, and staff costs could potentially be reduced. Furthermore, given the increased importance of zebrafish as a research organism within the scientific community, it is of great importance to bring solutions to fruition that aid in their monitoring and analysis. Permanent and accurate object tracking solutions definitely fit into this description.

5.3 Future Works

As mentioned in the discussion section of the results chapter, in order for an object moving in such an erratic and non-linear fashion as zebrafish to be accurately tracked, new object tracking algorithms must most certainly be developed. However, there are still numerous factors that could be pursued with the ambition of improving the groundwork laid in this thesis.

One such improvement would be to train the object detection model on a more diverse set of zebrafish samples to increase the likelihood that fish are detected in more frames. When analyzing the samples on which the detector was trained, it is evident that not many of them are representative of the conditions in the frames of the video samples. In the training material, there are few instances of fish turning around or being blurred, so the detector will be more likely to detect zebrafish that are moving slowly and perpendicular to the camera. By training the detector on additional samples where the fish are turning or motion-blurred, the trackers would most likely be able to track each fish for longer sequences.

When it comes to the implementation of the trackers, further improvements could also be pursued. The current implementations are more or less "vanilla" implementations without any particular hyperparameter optimizations. Since both trackers scored higher precision than recall, it is possible that hyperparameter optimization could be utilized to balance these scores and achieve overall better accuracy.

The physical setup in the lab is another area where improvements could be made. The video samples gathered in this thesis were collected by simply resting a smartphone on its side on the shelf next to the tank. This lead to the tank occupying far from the full video frame. By exploring camera mountings that would enable the tank to occupy closer to all of the video frame, the zebrafish would also become easier for the detector to detect. On the same topic, it would be beneficial to explore the impacts of mounting contrasting screens behind the tank in order to make the fish stand out more to their background, and thereby increasing likelihood of detection. This is something that was done in both [23][24] where they filmed from the a top-down angle with the tank resting on a LED-panel bottom for high-contrast images. Both of the physical improvements mentioned here must be done only after a careful evaluation of the impact on the fish has been made.

Lastly, some hardware improvements that could be explored are the introduction of an additional camera or improved computer hardware. Introducing an additional camera would open up the possibility of 3D tracking, which most likely would greatly improve the tracking abilities. However, without knowing the full extent of what such an implementation would look like, it seems very likely that it would increase the technical complexity of the system considerably. When it comes to computer hardware improvements, it would mainly be through GPU improvements where the greatest benefits in terms of frame rate would be achieved. The results obtained in this thesis were on a personal computer with a high-end consumer GPU. However, in both papers where ByteTrack and DeepSORT

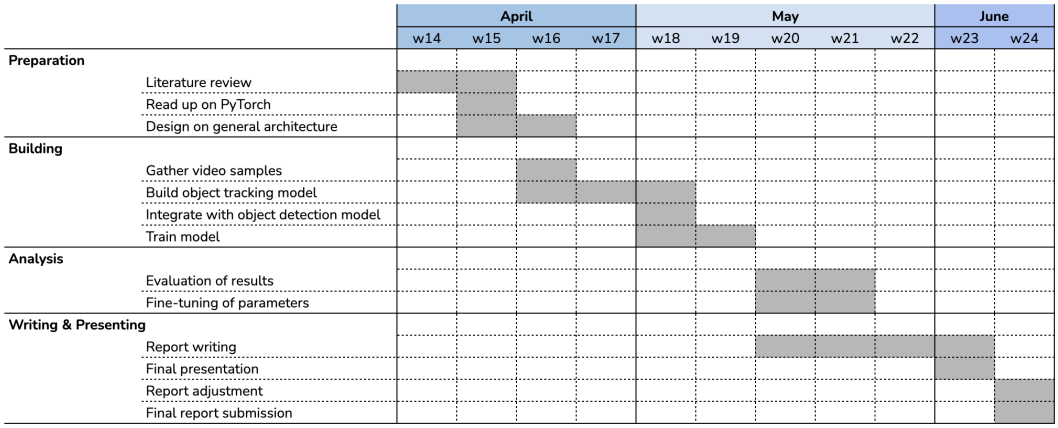


Figure 5.1: The preliminary project plan set up in the beginning of the project.

were introduced, they were tested on industry-grade GPUs, which can be theorized had a significant impact on their promising frame rate performances.

5.4 Review of Project Plan

As this project was initiated in the end of March 2025, a preliminary project plan was established as a guide for the duration of the project. The preliminary plan is shown in figure 5.1.

Without going into full detail on all departures during the actual project from the project plan, there are still many interesting discrepancies to cover. Starting from the top, the literature review was more of a process that ran in parallel with almost the full project, rather than only for the first two weeks. Although there was more focus on it during the first weeks, there were certainly more weeks in April and May in which the literature review was carried out.

The process of gathering video samples ended up taking place on two occasions, one in week 15 and one in week 19. The reason for this was that the first occasion was for the purpose of collecting video material for testing the implementation, and the second occasion was to gather video samples for the actual evaluation of the tracking models.

When it comes to the process of designing the architecture and the implementation of the detector and the trackers, there were also large discrepancies from the project plan. In the plan, this process was expected to take five weeks, while in reality it only required around three weeks. This was largely due to the detector already existing and being easy to integrate, the object tracking models being fairly easy to implement as well, and because no model training was required.

The last large discrepancy to mention is in regard to the report writing process. In the initial plan, my ambition was to focus heavily on literature review, design, and implementation during the first two-thirds of the project and to focus heavily on report writing in the last third of the project. However, after consulting with my supervisor, who recommended that I write small portions each day, I adjusted the writing process so that it ran in parallel with the entire project.

5.5 Learning Reflection

Naturally, after dedicating yourself to a project for two and a half months, numerous insights are acquired throughout the process. The most obvious knowledge gains come from the pure science behind object detection and tracking using computers. I have learned a lot about how they work from a detailed standpoint and what variations exist and for what purposes. In addition, I have also learned a lot about how to evaluate object tracking models in a scientific manner and how to process and evaluate testing results.

However, perhaps more importantly, I have learned a lot about leading my own project from start to finish. This is probably the largest undertaking to which I have committed myself and, therefore, I have also learned a lot about how to use my time efficiently, how to structure the project and how to process feedback from my supervisor. Much thought has gone into what information to include in the report, and at what stages, to ensure that the information follows a natural flow for the reader. The same can be said about the oral presentation where I wanted to ensure that the most important information from the project was presented while at the same time not bore the audience with technical details. Instead, the presentation focused on presenting the theory and results in a lightweight and interesting manner, which I think I achieved.

References

[1] Man-Technology-Environment research centre (MTM) - Örebro University;. Available from: https://www.oru.se/english/research/research-environments/ent/mtm/. (Cited on page 6.)

[2] What are PFAS? A guide to understanding chemicals behind nonstick pans, cancer fears;. Available from: https://eu.usatoday.com/story/news/2022/03/07/pfas-guide-chemicals/6652847001/. (Cited on page 6.)

[3] PFAS and Your Health;. Available from: https://www.atsdr.cdc.gov/pfas/about/?CDC_AAref_Val=https://www.atsdr.cdc.gov/pfas/health-effects/overview.html. (Cited on page 6.)

[4] How PFAS Impacts Your Health;. Available from: https://www.atsdr.cdc.gov/pfas/about/health-effects.html. (Cited on page 6.)

[5] G Lieschke PC. Animal models of human disease: zebrafish swim into view. Nat Rev Genet 8, 353–367. 2007. (Cited on page 6.)

[6] Storm E. YOLOv8 for Zebrafish – Object Detection for an Autonomous Fish Feeder. Örebro University, School of Science and Technology; 2024. (Cited on pages 6, 7, 16, and 41.)

[7] Kalman RE. A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering. 1960 03;82(1):35-45. Available from: https://doi.org/10.1115/1.3662552. (Cited on pages 9, 11, and 12.)

[8] Li Q, Li R, Ji K, Dai W. Kalman Filter and Its Application. In: 2015 8th International Conference on Intelligent Networks and Intelligent Systems (ICINIS); 2015. p. 74-7. (Cited on pages 9 and 10.)

[9] Kuhn HW. The Hungarian method for the assignment problem. Naval Research Logistics Quarterly. 1955;2(1-2):83-97. Available from: https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800020109. (Cited on pages 10, 11, and 12.)

[10] Isard M, Blake A. CONDENSATION—Conditional Density Propagation for Visual Tracking. International Journal of Computer Vision. 1998;29:5-28. (Cited on page 10.)

[11] Comaniciu D, Ramesh V, Meer P. Real-time tracking of non-rigid objects using mean shift. In: Proceedings IEEE Conference on Computer Vision and Pattern Recognition. CVPR 2000 (Cat. No.PR00662). vol. 2; 2000. p. 142-9 vol.2. (Cited on page 10.)

[12] Bolme DS, Beveridge JR, Draper BA, Lui YM. Visual object tracking using adaptive correlation filters. In: 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition; 2010. p. 2544-50. (Cited on page 10.)

[13] Henriques JF, Caseiro R, Martins P, Batista J. High-Speed Tracking with Kernelized Correlation Filters. IEEE Transactions on Pattern Analysis and Machine Intelligence. 2015;37(3):583-96. (Cited on page 10.)

[14] Bertinetto L, Valmadre J, Henriques JF, Vedaldi A, Torr PHS. Fully-Convolutional Siamese Networks for Object Tracking. In: Hua G, Jégou H, editors. Computer Vision – ECCV 2016 Workshops. vol. 9914. Springer International Publishing; 2016. p. 850-65. (Cited on page 11.)

[15] Chen X, Yan B, Zhu J, Wang D, Yang X, Lu H. Transformer Tracking. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR); 2021. p. 8126-35. (Cited on page 11.)

[16] Sun P, Jiang Y, Zhang R, Xie E, Cao J, Hu X, et al. TransTrack: Multiple-Object Tracking with Transformer. CoRR. 2020;abs/2012.15460. Available from: `https://arxiv.org/abs/2012.15460`. (Cited on page 11.)

[17] Ye B, Chang H, Ma B, Shan S, Chen X. Joint Feature Learning and Relation Modeling for Tracking: A One-Stream Framework. In: Avidan S, Brostow G, Cissé M, Farinella GM, Hassner T, editors. Computer Vision – ECCV 2022. Cham: Springer Nature Switzerland; 2022. p. 341-57. (Cited on page 11.)

[18] Zhang Y, Sun P, Jiang Y, Yu D, Weng F, Yuan Z, et al.. ByteTrack: Multi-Object Tracking by Associating Every Detection Box; 2022. Available from: `https://arxiv.org/abs/2110.06864`. (Cited on pages 11 and 12.)

[19] Wojke N, Bewley A, Paulus D. Simple online and realtime tracking with a deep association metric. In: 2017 IEEE International Conference on Image Processing (ICIP); 2017. p. 3645-9. (Cited on pages 11 and 12.)

[20] Bewley A, Ge Z, Ott L, Ramos F, Upcroft B. Simple online and realtime tracking. In: 2016 IEEE International Conference on Image Processing (ICIP); 2016. p. 3464-8. (Cited on page 12.)

[21] Mahalanobis PC. Reprint of: Mahalanobis, P.C. (1936) "On the Generalised Distance in Statistics.". Sankhya A. 20018;80(1):1-7. (Cited on page 12.)

[22] Xia C, Chon TS, Liu Y, Chi J, Lee J. Posture tracking of multiple individual fish for behavioral monitoring with visual sensors. Ecological Informatics. 2016;36:190-8. Available from: `https://www.sciencedirect.com/science/article/pii/S1574954116300887`. (Cited on page 13.)

[23] Barreiros MdO, Dantas DdO, Silva LCdO, Ribeiro S, Barros AK. Zebrafish tracking using YOLOv2 and Kalman filter. In: Scientific Reports. vol. 11; 2021. . (Cited on pages 13 and 43.)

[24] XU Z, Cheng XE. Zebrafish tracking using convolutional neural networks. Scientific Reports. 2017;7:42815. (Cited on pages 13 and 43.)

[25] Pérez-Escudero A, Vicente-Page RC Julián andHinz, Arganda S, de Polavieja GG. idTracker: tracking individuals in a group by automatic identification of unmarked animals. Nature Methods. 2014;11:743-8. (Cited on page 13.)

[26] Romero-Ferrero F, Bergomi MG, Hinz R, Heras FJH, de Polavieja GG. idtracker.ai: Tracking all individuals in large collectives of unmarked animals. CoRR. 2018;abs/1803.04351. Available from: `http://arxiv.org/abs/1803.04351`. (Cited on page 13.)

[27] Bernardin K, Stiefelhagen R. Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics. In: EURASIP Journal on Image and Video Processing; 2008. . (Cited on pages 14 and 15.)

[28] Ristani E, Solera F, Zou R, Cucchiara R, Tomasi C. Performance Measures and a Data Set for Multi-target, Multi-camera Tracking. In: Hua G, Jégou H, editors. Computer Vision – ECCV 2016 Workshops. Cham: Springer International Publishing; 2016. p. 17-35. (Cited on pages 14 and 15.)

[29] Luiten J, Osep A, Dendorfer P, Torr PHS, Geiger A, Leal-Taixé L, et al. HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking. CoRR. 2020;abs/2009.07736. Available from: https://arxiv.org/abs/2009.07736. (Cited on pages 14 and 15.)

[30] Milan A, Leal-Taixé L, Reid ID, Roth S, Schindler K. MOT16: A Benchmark for Multi-Object Tracking. CoRR. 2016;abs/1603.00831. Available from: http://arxiv.org/abs/1603.00831. (Cited on page 14.)

[31] HumanSignal. Labeling Evaluation;. Available from: https://humansignal.com/platform/#labeling-and-evaluation. (Cited on page 18.)

[32] ifzhang. ByteTrack;. Available from: https://github.com/ifzhang/ByteTrack.git. (Cited on page 19.)

[33] Jonathon Luiten AH. TrackEval; 2020. https://github.com/JonathonLuiten/TrackEval. (Cited on page 22.)

[34] Jordbruksverket. Registrering, godkännande och tillstånd för verksamhet med vattenlevande djur;. Available from: https://jordbruksverket.se/djur/ovriga-djur/fiskar-kraftdjur-och-blotdjur/fiskar-kraftdjur-och-blotdjur-i-kommersiell-verksamhet/registrering-godkannande-och-tillstand. (Cited on page 42.)